Reinforcement Learning for Within-Visual-Range Air Combat with Curriculum Learning and Dynamic Opponent Pooling

Yue Yu[1], Yuhang Wang[1], Zhe Zhang[2], Bei Lu[1], Qifu Li[1*]

[1*]School of Aeronautics and Astronautics, Shanghai Jiao Tong University, Shanghai, China.
[2]Chinese Flight Test Establishment, Xi'an, China.

*Corresponding author(s). E-mail(s): qifuli@sjtu.edu.cn;

Abstract

Achieving stable training and robust generalization in autonomous air combat remains a significant challenge for unmanned combat aircraft. This study proposes a reinforcement learning framework for within-visual-range dogfights in a six-degree-of-freedom air combat simulation environment based on JSBSim. A proximal policy optimization-based architecture is developed, featuring curriculum learning and a dynamic opponent pool. The curriculum progressively shifts the learning objective from stable flight control to tactical air-combat behavior, while gradually increasing opponent threat levels to alleviate sparse-reward exploration difficulty. The dynamic opponent pool combines protected opponents with competitive historical policy snapshots, improving adversarial diversity and mitigating policy overfitting during self-play. Engagement simulations demonstrate that the proposed framework achieves stable convergence and improved tactical performance, with the best proposed model improving the win rate over the baseline by 55.2 percentage points.

Keywords: reinforcement learning, within-visual-range air combat, dogfight, curriculum learning, dynamic opponent pool

1 Introduction

Recent advances in artificial intelligence have driven air combat away from rule-based systems and toward data-driven approaches [1]. Dogfight scenarios are especially challenging because they require aircraft to interpret high-dimensional states and generate continuous control commands at high frequency under rapidly changing conditions. Reinforcement learning (RL) improves behavior through repeated trial-and-error in the environment, making it well suited to dynamic adversarial settings. The potential of RL for dogfight has been supported by recent studies. A representative example is the 2020 Defense Advanced Research Projects Agency (DARPA) AlphaDogfight Trials (ADT), in which an artificial intelligence (AI) agent defeated an experienced human pilot in a one-on-one within-visual-range (WVR) dogfight [2]. This result suggested that AI could achieve competitive performance in tactical decision-making and maneuvering control, and it further stimulated research on autonomous unmanned combat aircraft.

Even so, robust performance in dogfight scenarios remains difficult to achieve. Specifically, the three major challenges are the training instability inherent in high-dimensional continuous control, the exploration difficulty under sparse rewards, and the policy overfitting caused by self-play.

The high-dimensional and continuous nature of aircraft dynamics often leads to convergence difficulty during training. To alleviate this, many studies have adopted hierarchical architectures to decouple complex control tasks. For instance, at the low-level maneuvering layer, RL-based controllers have outperformed conventional proportional-integral-derivative (PID) controllers in response speed and tracking accuracy, while also showing stronger adaptability to severe wind disturbances and aggressive maneuvers [3–7]. Similarly, to address the curse of dimensionality and the credit assignment problem in multi-aircraft cooperation, hierarchical architectures and value decomposition methods have been used [8–11]. Methods including hierarchical control and gain scheduling have also been proposed to address convergence difficulty and oscillatory behavior in end-to-end control [12]. Notably, one study based on JSBSim, an open-source flight dynamics model widely used for aircraft simulation [13], demonstrated the effectiveness of a hierarchical RL architecture in experiments against a human pilot [2]. While hierarchical methods simplify the learning process, they often require manual design and may introduce sub-optimalities at the interfaces between decision-making and control layers. Consequently, end-to-end control frameworks are desirable for their potential to learn globally optimal behaviors, though achieving stable convergence remains challenging. To improve end-to-end training stability and sample efficiency, researchers have proposed progressive multi-objective strategy optimization methods [14], as well as model-based reinforcement learning [15]. Furthermore, sequence models such as Transformers and long short-term memory networks (LSTMs) have been integrated to enhance long-horizon decision-making and state estimation under partial observability [16–19].

Beyond control stability, dogfight tasks typically suffer from exploration difficulties due to sparse reward signals. To alleviate this issue and improve RL performance, curriculum learning and imitation learning have been actively explored [20, 21]. The performance of imitation learning remains limited. For instance, it has been observed

that even when facing non-adversarial opponents performing basic maneuvers such as level flight, circling, or serpentine patterns, the shooting success rates achieved through imitation learning still fall short of those seen in expert demonstrations [21]. Studies employing curriculum learning progressively expose agents to increasingly challenging scenarios, such as gradually reducing initial tactical advantages or introducing more competitive opponents [19, 22–24]. Therefore, curriculum learning serves as an effective approach to smoothly guiding the exploration process and overcoming the sparse reward challenge in complex dogfight scenarios.

Within a reinforcement learning framework, improving generalization requires addressing policy overfitting and cyclic dominance that often arise during self-play [25, 26]. To promote gradual and stable performance improvement, one study employed simulated annealing to design a dynamic opponent-pool update rule: a new policy was added to the opponent pool only when its win rate exceeded 50%, the oldest policy was removed, and the sampling distribution was progressively shifted toward stronger opponents [26]. Another study approached the problem from the perspective of population diversity, introducing a population-entropy regularization term to enlarge the KL divergence among policies in the opponent pool [25]. While this entropy-based method improved the generalization of the main agent, its validation was limited to a low-fidelity three-degree-of-freedom point-mass model with a discrete action space. Building on these insights, designing a robust dynamic opponent pool that effectively balances diversity and competence in high-fidelity, continuous-action environments remains essential. Furthermore, a challenge in such frameworks is the cold-start problem, which concerns the selection of appropriate initial opponents to kickstart the learning process. Whether utilizing expert strategies [21] or pre-trained models [25] as baseline adversaries, the competence and diversity of these initial opponents can significantly influence the convergence stability and the ultimate proficiency of the agent.

While RL shows great promise for air combat, convergence under sparse rewards, cyclic dominance, and overfitting in self-play continue to be important open problems. These intertwined challenges form the central focus of the present study.

This work presents a reinforcement learning framework for one-on-one within-visual-range gun-based dogfights, and evaluates it in a six-degree-of-freedom (6-DOF) air combat environment built on JSBSim [13]. The environment models an F-16 aircraft, and adopts engagement criteria consistent with the DARPA setting reported in [2]. Proximal policy optimization (PPO) is used as the underlying learning algorithm. To address the sparse rewards and overfitting tendencies commonly encountered in self-play air combat training, the framework incorporates a curriculum learning framework and a dynamic opponent pool.

The main contributions of this work are summarized as follows:

- A curriculum learning framework is developed, consisting of an objective curriculum that transitions from stable flight control to air-combat objectives, and an opponent curriculum in which adversary difficulty increases progressively over training.
- A dynamic opponent pool is introduced to maintain policy diversity through evaluation during training. In addition, a goalpoint-based RL opponent is incorporated to mitigate the cold-start problem.

The remainder of this paper is organized as follows. Section 2 presents the mathematical formulation of the air combat task and the construction of the simulation environment. Section 3 describes the PPO-based training framework, including the curriculum and the dynamic opponent-pool update strategy. Section 4 evaluates the tactical performance of the trained agent across multiple engagement scenarios. Section 5 concludes the paper.

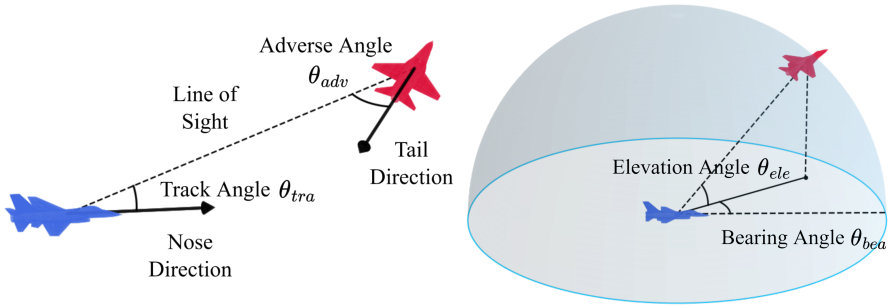
2 Environment Setup

2.1 Air Combat Formulation

WVR air combat is a dynamic and nonlinear adversarial process. In the one-on-one WVR gun dogfight scenario considered in this study, two aircraft execute complex tactical maneuvers in a continuous three-dimensional space in order to secure a favorable firing position behind the opponent. Under a gun-only weapon configuration, the aircraft must avoid incoming fire while closing on the opponent and satisfying range constraints to achieve a valid kill. This setting requires the policy to handle long-horizon spatiotemporal interactions under rapidly changing engagement conditions.

Under a fixed opponent policy, the dogfight process can be approximated from the perspective of the agent as a Markov decision process (MDP), represented by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$. Here, the state space $\mathcal{S}$ and action space $\mathcal{A}$ define the interaction interface between the aircraft and the environment, while $\mathcal{P}$ denotes the transition function and $\mathcal{R}$ denotes the reward function.

The state space $\mathcal{S}$ provides the agent with a structured description of the current engagement and the aircraft's own flight state. In this work, the observation is represented as a continuous vector. Table 1 summarizes the observation variables, while their full definitions and bounds are provided in Table A1. The angles of relative geometry are shown in Figure 1.



(a) Track angle and adverse angle

(b) Elevation angle and bearing angle

Fig. 1 Definition of angular variables in air combat geometry.

Instead of using a hierarchical control architecture based on high-level waypoints or attitude-command decomposition, this work adopts a mapping from observations

Table 1 Definitions of key state variables in the observation space.

Category	Variable symbol and meaning
Ego Basic States	h, Altitude u, v, w, Body-axis velocities p, q, r, Body-axis angular velocities θ, ϕ, Pitch and roll angles
Relative Geometry	D, Relative distance V_c, Closure rate θ_{tra}, Track angle θ_{adv}, Adverse angle θ_{bea}, Bearing angle θ_{ele}, Elevation angle
Opponent Features	$\mathbf{x}_{oppo}$, Ego basic states and relative geometry from the opponent's perspective
Action Variables	$\delta_a, \delta_e, \delta_r$, Aileron, elevator, and rudder commands δ_T, Throttle command

directly to pilot inputs. Accordingly, the action space $\mathcal{A}$ is defined as a four-dimensional continuous vector, whose components correspond to normalized target roll rate, target pitch rate, target yaw rate, and throttle commands.

The transition function $\mathcal{P}(s_{t+1} \mid s_t, a_t)$ is governed by the equations of motion evaluated by JSBSim. Episode termination is jointly determined by weapon effectiveness, flight-safety constraints, and a maximum engagement duration. Specifically, an episode terminates when an aircraft falls within the opponent's gun engagement zone and its health points (HP) are reduced to zero. In addition, if an aircraft crashes into the ground during aggressive maneuvering, that is, when the altitude satisfies $h \leq 0$, the episode is forcibly terminated. Finally, to prevent prolonged circling at long range or static stalemates, the simulator imposes a maximum episode length of 60 seconds of physical time. Once this limit is reached, the engagement is truncated and scored as a draw.

2.2 Simulation Environment

This work constructs the training environment using JSBSim, an open-source high-fidelity flight dynamics engine. The open-source project Gym-jsbsim [27] provides a viable approach to wrapping JSBSim into a Gymnasium-compatible environment. During simulation, the environment state is updated at a frequency of 60 Hz.

The core dynamics of the system are governed by the real-time integration of a set of ordinary differential equations. The translational equations of motion can be written as

$$\begin{cases} \dot{u} = rv - qw + \frac{1}{m}\left(W_x + F_x^{(A)} + F_x^{(T)}\right) \\ \dot{v} = -ru + pw + \frac{1}{m}\left(W_y + F_y^{(A)} + F_y^{(T)}\right) \\ \dot{w} = qu - pv + \frac{1}{m}\left(W_z + F_z^{(A)} + F_z^{(T)}\right) \end{cases}, \tag{1}$$

where u, v, w and p, q, r denote the translational velocities and angular rates along the three body axes, respectively; m is the aircraft mass; $W_{x,y,z}$ are the gravity components; and $F^{(A)}$ and $F^{(T)}$ represent the resultant aerodynamic forces and engine thrust, respectively. Correspondingly, the rotational equations of motion are given by

$$\begin{cases} \dot{p} = (C_1 r + C_2 p)q + C_3 \mathcal{L} + C_4 \mathcal{N} \\ \dot{q} = C_5 pr - C_6(p^2 - r^2) + C_7 \mathcal{M} \\ \dot{r} = (C_8 p - C_2 r)q + C_4 \mathcal{L} + C_9 \mathcal{N} \end{cases}, \tag{2}$$

where $\mathcal{L}$, $\mathcal{M}$, and $\mathcal{N}$ denote the total rolling, pitching, and yawing moments generated by aerodynamic and propulsive effects. The coefficients C_1 to C_9 are determined entirely by the rigid-body inertia properties of the aircraft, which are defined as follows.

Let $Ox_B y_B z_B$ denote the body-fixed frame, whose origin O is located at the aircraft centre of gravity, with the x_B axis pointing forward along the fuselage reference line, the y_B axis pointing to the right wing, and the z_B axis pointing downward. The inertia tensor of the aircraft expressed in this frame is

$$\mathbf{J} = \begin{bmatrix} I_{xx} & -I_{xy} & -I_{xz} \\ -I_{xy} & I_{yy} & -I_{yz} \\ -I_{xz} & -I_{yz} & I_{zz} \end{bmatrix}, \tag{3}$$

where (x, y, z) denote the coordinates of a mass element $\mathrm{d}m$ in the body frame, the diagonal entries

$$I_{xx} = \int \left(y^2 + z^2\right) \mathrm{d}m, \qquad I_{yy} = \int \left(x^2 + z^2\right) \mathrm{d}m, \qquad I_{zz} = \int \left(x^2 + y^2\right) \mathrm{d}m \tag{4}$$

are the moments of inertia about the x_B, y_B, and z_B axes, respectively, and the off-diagonal entries

$$I_{xy} = \int xy \, \mathrm{d}m, \qquad I_{xz} = \int xz \, \mathrm{d}m, \qquad I_{yz} = \int yz \, \mathrm{d}m \tag{5}$$

are the corresponding products of inertia. All six quantities are taken about the centre of gravity and have units of slug $\cdot$ ft^2. Since the aircraft is symmetric about its x_B–z_B plane, the products of inertia I_{xy} and I_{yz} vanish, so that I_{xz} is the only non-zero off-diagonal element of $\mathbf{J}$. As $\mathbf{J}$ is symmetric and positive definite, the quantity $\Gamma =$

$I_{xx}I_{zz} - I_{xz}^2$ is strictly positive. With these definitions, the coefficients are expanded as

$$\begin{aligned} C_1 &= \frac{(I_{yy} - I_{zz})I_{zz} - I_{xz}^2}{\Gamma}, \quad C_2 = \frac{(I_{xx} - I_{yy} + I_{zz})I_{xz}}{\Gamma}, \\ C_3 &= \frac{I_{zz}}{\Gamma}, \quad C_4 = \frac{I_{xz}}{\Gamma}, \quad C_5 = \frac{I_{zz} - I_{xx}}{I_{yy}}, \quad C_6 = \frac{I_{xz}}{I_{yy}}, \\ C_7 &= \frac{1}{I_{yy}}, \quad C_8 = \frac{(I_{xx} - I_{yy})I_{xx} + I_{xz}^2}{\Gamma}, \quad C_9 = \frac{I_{xx}}{\Gamma}. \end{aligned} \tag{6}$$

For the F-16 model used in this work, these inertia quantities are updated by JSBSim [13] at every integration step from the airframe inertia tensor together with the pilot and fuel point masses, and therefore depend on the instantaneous loading state. At the loading state used for the initial conditions of the engagements, that is, with an initial fuel load of 3000 lb giving a total weight of 20,630 lb and $m = 641.2$ slugs, the moments and the product of inertia, expressed in slug $\cdot$ ft^2, are

$$I_{xx} = 1.229 \times 10^4, \quad I_{yy} = 5.711 \times 10^4, \quad I_{zz} = 6.707 \times 10^4, \quad I_{xz} = 1.060 \times 10^3. \tag{7}$$

The values given in Eq. (7) vary with the fuel state; when the fuel is exhausted ($m = 548.0$ slugs, that is, 17,630 lb), they become $I_{xx} = 9.50 \times 10^3$, $I_{yy} = 5.682 \times 10^4$, $I_{zz} = 6.411 \times 10^4$, and $I_{xz} = 9.46 \times 10^2$ slug $\cdot$ ft^2. The mass m in Eq. (1) is updated in the same manner.

JSBSim includes an inner-loop fly-by-wire (FBW) architecture. Unlike studies that directly output control-surface deflection commands, the continuous action vector output by the agent, $\mathbf{u}_{cmd} = [u_{roll}, u_{pitch}, u_{yaw}, u_T]^T$, represents pilot-level stick and throttle inputs that serve as target rate and G-load setpoints. Given the current flight state $\mathbf{x}$, the FBW system maps these reference signals into the actual actuator-level surface deflections and engine commands, denoted by $[\delta_a, \delta_e, \delta_r, \delta_T]^T$. This design is consistent with the human–machine interaction logic of a real cockpit and also allows the reinforcement learning agent to benefit implicitly from the built-in stability augmentation loops, thereby focusing on higher-level tactical decision-making rather than high-frequency disturbance rejection at the control-surface level.

The weapon system and engagement criteria used in this work are inspired by the publicly available rules of the DARPA ADT. The environment defines a weapons engagement zone (WEZ) in front of the aircraft nose, shown in Figure 2. Let the position of the ego aircraft be $\mathbf{p}_{agt}$ and that of the opponent aircraft be $\mathbf{p}_{oppo}$. The relative line-of-sight vector is defined as $\mathbf{d} = \mathbf{p}_{oppo} - \mathbf{p}_{agt}$, and the relative Euclidean distance is $D = \|\mathbf{d}\|_2$. If the unit nose-direction vector of the ego aircraft is denoted by $\mathbf{v}_{nose}$, then the track angle θ_{tra} is computed from the inner product as

$$\theta_{tra} = \arccos\left(\frac{\mathbf{d} \cdot \mathbf{v}_{nose}}{\|\mathbf{d}\|_2}\right). \tag{8}$$

The opponent is considered to be inside the WEZ only when it lies within the effective firing range $[D_{\min}, D_{\max}]$ and inside a cone of half-aperture angle $\theta_{\max}$ around the

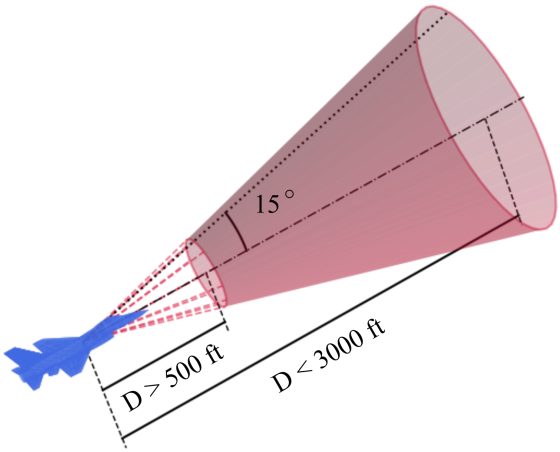


Fig. 2 Definition of the weapons engagement zone.

gun line. This condition can be expressed by the indicator function $\mathbb{W}_{WEZ}(t)$:

$$\mathbb{W}_{WEZ}(t) = \begin{cases} 1, & \text{if } D_{\min} \leq D(t) \leq D_{\max} \text{ and } \theta_{tra}(t) \leq \theta_{\max} \\ 0, & \text{otherwise} \end{cases}, \tag{9}$$

where $D_{\min}$ and $D_{\max}$ denote the near and far boundaries of the effective firing range and $\theta_{\max}$ denotes the maximum admissible off-boresight angle. These three thresholds are not free parameters of the learning problem, since they describe the geometry of a valid firing solution rather than the behaviour of the agent. They were determined by considering both the practical characteristics of within-visual-range gun engagements and the publicly released rules of the DARPA ADT. In this work they are set to $D_{\min} = 500$ ft, $D_{\max} = 4500$ ft and $\theta_{\max} = 15°$. The near boundary corresponds to the minimum separation at which a tracking solution can still be established, and it rules out an unrealistic point-blank firing condition; the far boundary corresponds to the range beyond which the dispersion of the gun and the loss of lead accuracy make hits improbable at the closing speeds of the modelled aircraft; and the cone angle follows the ADT criterion, which requires a close alignment between the target and the gun line.

Both aircraft enter an engagement with an initial health of $\mathrm{HP}_0 = 3$. When $\mathbb{W}_{WEZ}(t) = 1$, the agent applies continuous damage to the opponent at the current time step, and the decay of the opponent's health follows

$$\mathrm{HP}_{oppo}(t + \Delta t) = \max\left(0, \mathrm{HP}_{oppo}(t) - \lambda(D) \cdot \mathbb{W}_{WEZ}(t) \cdot \Delta t\right), \tag{10}$$

where $\lambda(D) = \lambda_0 \left(D_{\max} - D(t)\right) / \left(D_{\max} - D_{\min}\right)$ is the prescribed per-unit-time gun damage rate and $\lambda_0 = 1$ HP/s is its value at the near boundary. The damage rate therefore decreases linearly with range and vanishes at the far boundary of the WEZ, so that a target held at the minimum range is destroyed within 3 s, whereas a target

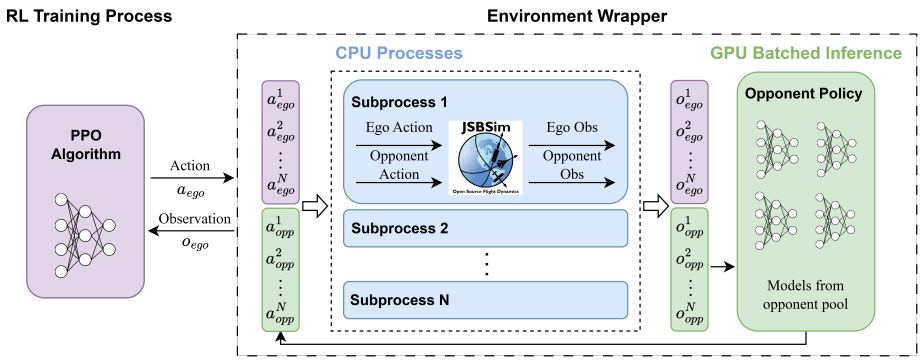


Fig. 3 Multiprocessing parallel acceleration architecture.

engaged near the far boundary survives correspondingly longer for the same amount of tracking time.

At the same time, aerodynamic data, FBW computation, and combat logic are highly decoupled within the JSBSim framework, so the proposed simulation architecture has good extensibility. By replacing the aerodynamic database and model configuration, the environment can be extended to other aircraft types, such as the F-15, providing a basis for future studies on multi-aircraft and asymmetric air combat.

2.3 Parallel Sampling Architecture

In RL, environment sampling throughput is often the key bottleneck that limits training efficiency. From the perspective of the agent, the opponent policy should be treated as part of the environment transition. However, in practical implementation, directly coupling the opponent network into each simulation time step can substantially reduce training speed. Specifically, JSBSim flight dynamics simulation relies on serial floating-point computation on the CPU, whereas opponent policy inference is better suited to parallel tensor computation on the GPU. If CPU-based dynamics integration and single-sample neural network forward passes are executed alternately at every time step, the system suffers not only from frequent context switching, but also from the inability to exploit the batching advantage of the GPU. As a result, sampling efficiency is significantly degraded by computational serialization.

To address this issue, this work designs a multiprocessing parallel acceleration architecture based on heterogeneous-computing decoupling, whose core idea is to separate dynamics simulation from neural network inference. Figure 3 shows the detail. First, the base environment is refactored into a pure JSBSim dynamics container with no embedded neural network modules. It exposes a state-action interface for both the ego aircraft and the opponent aircraft. Second, by combining CPU multiprocessing with GPU batched inference, a vectorized multi-process environment wrapper is constructed. Each subprocess independently executes flight dynamics simulation. At every interaction step the opponent observations from all sub-environments are collected, concatenated, and sent to the GPU for batched forward inference through the fixed opponent policy network. The resulting opponent actions are then distributed back

Table 2 Measured environment sampling throughput on the workstation described in Section 4. Every entry is the median of three runs of 20000 decision steps following a warm-up of 200 steps, and both throughput columns are expressed in steps per second. A decision step is one interaction of the agent with the environment (10 Hz) and comprises six JSBSim integration steps (60 Hz). The two serial implementations differ only in how the same per-step opponent inference is deployed, namely in a single process or replicated in every worker.

Implementation (dynamics processes)	Opponent policy inference	Decision steps s^{-1}	Integration steps s^{-1}
Serial, single process (1)	One forward pass per decision step	1228	7367
Serial, one policy instance per worker (30)	One forward pass per decision step, in every worker	2343	14058
Proposed decoupled sampling (30)	One batched forward pass per parallel step	8396	50378

to the corresponding CPU processes for the next simulation step. Finally, through an external interface abstraction, the wrapper hides the opponent observations and actions from the learning algorithm and exposes only the state and action interfaces of the ego agent. Therefore, under a fixed opponent policy, the overall training procedure remains externally equivalent to a standard single-agent Markov decision process.

This decoupled architecture effectively leverages heterogeneous computing resources. Table 2 reports the sampling throughput measured on the workstation described in Section 4, where the compared implementations were driven from the same initial-condition distribution, the same opponent pool and the same fixed command sequence, so that the sampling architecture is the only difference between them. The environment integrates the flight dynamics at 60 Hz while the agent interacts with it at 10 Hz, so that one decision step comprises six integration steps; both rates are reported in order to remove any ambiguity about the unit of a step. At an equal number of processes, decoupling raises the aggregate sampling rate from 2343 to 8396 decision steps per second, that is by a factor of 3.6, and against a single-process serial implementation, which is the limiting case of the same serialization, the factor is 6.8.

The improvement is not obtained by adding arithmetic capacity. A single forward pass through the opponent policy performs approximately 1.5×10^6 floating-point operations, so that even at the highest measured rate the aggregate inference load remains of the order of $10^{10}\,\mathrm{FLOP\,s^{-1}}$, more than three orders of magnitude below the arithmetic capability of the GPU used in the experiments. The serialized implementations are limited instead by the latency of short and frequent calls: while every CPU core is occupied by flight-dynamics integration, each decision step pays separately for launching the inference, synchronizing the worker threads and transferring the observation and action tensors between host and device. Collecting the observations of all sub-environments into one batch replaces thirty such calls per parallel step with a single one, so that this overhead is amortized over the whole batch and the resulting batch is large enough to occupy the arithmetic units of the accelerator. Conversely, the flight dynamics themselves are parallelized over processes rather than over threads, because the JSBSim integration is a scalar and branch-intensive workload that does not release the interpreter lock. This decoupling therefore converts a

latency-bound sampling procedure into a throughput-bound one, which provides the computational support for the training runs reported below.

3 Methods

PPO [28] is a representative Actor–Critic policy optimization method and has been widely applied to decision-making tasks such as robotic control, flight control, and autonomous driving. The core idea of PPO lies in formulating a constrained policy update objective that prevents the new policy from deviating excessively from the old policy.

Let the current policy parameters be θ and the old policy parameters be θ_{old}. For a state s_t and action a_t, the importance sampling ratio is defined as

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}. \tag{11}$$

PPO adopts a clipped surrogate objective function:

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right], \tag{12}$$

where $\hat{A}_t$ denotes the advantage function estimate and ϵ is the clipping coefficient. By clipping the importance sampling ratio, this objective restricts the magnitude of the policy change in a single iteration, improving training stability. Meanwhile, a value function network is used to estimate state values and is jointly optimized with the policy network. Combining the policy term, the value term, and an entropy regularization term, the overall optimization objective can be written as

$$L(\theta, \phi) = L^{\text{clip}}(\theta) - c_1 L^{\text{value}}(\phi) + c_2 H(\pi_\theta), \tag{13}$$

where $L^{\text{value}}(\phi)$ represents the value function loss, ϕ is the value network parameters, $H(\pi_\theta)$ is the policy entropy. The weight coefficients c_1 and c_2 are used to balance the contributions of the value-function loss and the entropy regularization, respectively. In practice, c_1 is chosen such that the value loss has a comparable scale to the policy objective and does not dominate the policy update, while c_2 is usually set to a relatively small value to encourage sufficient exploration without preventing policy convergence. These coefficients are typically selected empirically according to the training stability and final performance, and can be tuned through validation experiments.

To estimate the advantage, we employ generalized advantage estimation (GAE), which takes the form

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \qquad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \tag{14}$$

where V represents the value network, γ is the discount factor and λ is the advantage estimation parameter.

Compared with standard policy gradient methods, it constrains the policy update magnitude, which helps mitigate the instability caused by drastic policy oscillations during training. Therefore, PPO is well suited as the foundational algorithmic framework for air combat control policy learning.

3.1 Baseline

This work adopts PPO as the baseline reinforcement learning algorithm and implements it based on Stable-Baselines3 [29].

We pre-construct a pool of several level-flight opponents. These opponents are all trained using PPO and learn stable flight policies under different reward functions. They are designed to maintain sustained flight under randomly initialized headings without losing control or crashing. These opponents possess only basic flight capability and cannot track targets. Compared with stationary targets or a scripted opponent, this design introduces more realistic flight-dynamics perturbations during baseline training and improves the effectiveness of policy learning.

These level-flight opponents are trained using different combinations of several basic reward terms, which are listed in Table A2.

During baseline training, the agent randomly samples opponents from this fixed opponent pool. In practice, the baseline model can acquire basic target-tracking and kill capabilities against opponents from the training set. Nevertheless, when evaluated against unseen opponents or in self-play tests, the baseline model exhibits weak generalization. In many cases, it fails to locate the target and initiate an attack against opponents outside the training set. In some situations, it even crashes. To address this issue, this work introduces a curriculum learning strategy and a dynamic opponent pool.

3.2 Curriculum Learning

Curriculum learning is commonly used to mitigate the training difficulty caused by sparse rewards. It constructs a task sequence that progresses from simple to complex. In this section, we propose a curriculum learning method composed of an objective curriculum and a dynamic opponent curriculum. The former employs two progressively designed reward functions to ensure that the agent first acquires stable flight control and then gradually develops air combat tactics. The latter adaptively adjusts opponent difficulty to provide progressively increasing game pressure for policy improvement. This design is intended to improve training stability and convergence efficiency.

3.2.1 Objective Curriculum

We implement the objective curriculum by progressively adjusting the reward function. The objective curriculum is structured in two successive stages, where the agent first masters stable flight and then learns air combat skills. The first stage includes only

reward terms related to flight quality and does not involve relative engagement geometry or firing-related indicators. The second stage introduces reward terms related to relative geometry and gun engagement, so that the agent can learn tactical positioning and evasive maneuvering.

In both stages the reward is composed of a small number of normalized terms, each of which represents one requirement of the task and is scaled by a weight, so that the contribution of the individual requirements can be analysed and adjusted separately. The general expressions of the terms of the two stages are given in Table 3 and Table 4, respectively, together with the notation used for their thresholds and weights. The values assigned to these parameters, and the procedure by which they were calibrated, are described in Section 3.2.2.

Table 3 Stage-1 reward terms and their general expressions.

Reward term	Mathematical expression
Aileron command penalty	$r_{sa} = w_{sa} \dfrac{s_{sa}}{s_{sa} + \lvert\delta_a - \delta_a^*\rvert}$
Throttle command penalty	$r_{st} = w_{st} \dfrac{s_{st}}{s_{st} + \lvert\delta_T - \delta_T^*\rvert}$
Roll-angle penalty	$r_{sr} = w_{sr} \dfrac{s_{sr}}{s_{sr} + \lvert\phi - \phi^*\rvert}$
Action smoothness penalty	$r_{ap} = w_{ap}\,(\Psi_{ap}(t) - \Psi_{ap}(t-1))$, $\Psi_{ap}(t) = -\max_j \dfrac{\overline{\lvert\Delta u_j\rvert}_{L_{ap}}}{u_{j,ub}}$
Altitude stability penalty	$r_{alt} = w_{alt}\,(\Psi_{alt}(t) - \Psi_{alt}(t-1))$, $\Psi_{alt}(t) = -\dfrac{\overline{\lvert\Delta h\rvert}_{L_{alt}}}{h_{ub}}$

The subscript "ub" in Table 3 denotes the upper bound of the corresponding state variable, that is, $u_{j,ub} = 1$ for the normalized surface commands and $h_{ub} = 85000$ ft for the altitude; δ_a^*, δ_T^* and ϕ^* are the reference values of the aileron input, the throttle input and the roll angle, respectively; $w_\bullet$ and $s_\bullet$ denote the weight and the scaling factor of a term; and $\overline{\lvert\Delta x\rvert}_L$ denotes the mean absolute step-to-step change of the quantity x over the preceding L decision steps.

The first three terms are asymptotic error potentials. Each of them attains its maximum $w_\bullet$ when the corresponding variable equals its reference value and decays as the deviation grows, with $s_\bullet$ setting the deviation at which half of the maximum is retained. These terms therefore encode the requirement of a trimmed and level flight condition rather than a specific manoeuvre: the aileron and roll terms suppress unnecessary lateral control activity and residual bank, while the throttle term concentrates the engine thrust around the cruise setting, which prevents both the loss of airspeed that would reduce the available control authority and the sustained over-throttle condition that would accompany an aggressive energy strategy.

The two remaining terms are potential-difference terms, and they penalize the variation of the control activity and of the vertical motion rather than their absolute level. Since the potentials are defined as the windowed mean absolute rates of change, a steady manoeuvre leaves Ψ almost unchanged and therefore produces almost

no penalty, whereas rapid reversals of the control surfaces or abrupt changes of the vertical rate produce a penalty proportional to the corresponding increment. This formulation thus acts as a smoothness regularizer that does not discourage sustained manoeuvring, and, because the potentials depend only on the variation of the state, it does not bias the policy towards any particular engagement geometry. The weights of the two terms differ by more than four orders of magnitude, which reflects the different scales of the quantities involved: the normalized control deviation is of order 10^{-1} to 10^{0}, whereas the normalized altitude increment of a typical manoeuvre is of order 10^{-4}, so that a common weight would render the altitude term numerically inactive.

Table 4 Stage-2 reward terms and their general expressions.

Reward term	Mathematical expression
Relative position reward	$r_{rp} = w_{rp}\left[\left(\hat{\theta}_{tra} - 2\right)\sigma\left(\hat{\theta}_{adv}; k_a, \hat{\theta}_a\right) - \hat{\theta}_{tra} + 1\right]$
Closure rate reward	$r_{cr} = w_{cr}\dfrac{V_c}{V_{ref}}\left[1 - \sigma\left(\hat{\theta}_{adv}; k_a, \hat{\theta}_a\right)\right]\sigma\left(D; k_D, D_c\right)$
Aiming reward	$r_{gb} = \Gamma_B(D)\left[1 - \sigma\left(\hat{\theta}_{tra}; k_g, \hat{\theta}_g\right)\right]$
Opponent aiming penalty	$r_{gr} = -\Gamma_R(D)\,\sigma\left(\hat{\theta}_{adv}; k_r, \hat{\theta}_r\right)$
Low-altitude penalty	$r_{deck} = -w_{deck}\left[1 - \sigma\left(h; k_h, h_{deck}\right)\right]$

In Table 4, $\sigma(x; k, x_0) = 1/(1 + e^{-k(x-x_0)})$ is the logistic function, in which k controls the slope of the transition and x_0 its centre, and the normalized angles $\hat{\theta}_{tra} = \theta_{tra}/\pi$ and $\hat{\theta}_{adv} = \theta_{adv}/\pi$ are used so that all angular thresholds are expressed on a common scale. The adverse angle θ_{adv} measures the angular advantage of the ego aircraft: it is zero when the ego aircraft is directly behind the opponent, and it increases to π when the opponent has reversed the situation and the ego aircraft lies in front of it. A small adverse angle therefore corresponds to an offensive situation, and a large one to a defensive situation.

The aiming reward and the opponent aiming penalty are weighted by the functions $\Gamma_B(D)$ and $\Gamma_R(D)$, which represent the range dependence of the firing opportunities of the two aircraft. Both are band-pass functions of range of the form

$$\Gamma_i(D) = \beta_i \min\left\{\sigma\left(D; k_{i,1}, D_{i,1}\right),\ 1 - \sigma\left(D; k_{i,2}, D_{i,2}\right)\right\}, \qquad i \in \{B, R\},, \tag{15}$$

where the two logistic factors describe the near and the far boundary of the effective firing band and β_i is the value of the function inside that band. A firing solution is thus treated as being most valuable in the intermediate range band, where a tracking solution can be established and sustained, and as being less valuable at very short range, where the geometry changes rapidly and an overshoot is imminent, and beyond the far boundary, where hits become improbable.

The relative position reward r_{rp} is the primary shaping term for the angular advantage. It is bounded in $[-w_{rp}, w_{rp}]$, and it attains $+w_{rp}$ when the ego aircraft is simultaneously behind the opponent ($\hat{\theta}_{adv} \to 0$) and pointing its nose at it ($\hat{\theta}_{tra} \to 0$), whereas it attains $-w_{rp}$ when the opponent has established the same advantage over the ego aircraft, because then $\sigma \to 1$ and the term reduces to -1 independently of $\hat{\theta}_{tra}$. The logistic factor with slope k_a and centre $\hat{\theta}_a$ separates the offensive and the defensive regime smoothly, so that the term provides a continuous gradient over the whole engagement instead of a binary assessment of the relative geometry.

The closure rate reward r_{cr} complements the angular term by shaping the approach. It is proportional to the closure rate normalized by V_{ref}, so that reducing the distance is rewarded while the ego aircraft is in the offensive regime, and it is gated by two logistic factors: the first one removes the reward when the ego aircraft is not behind the opponent, which prevents the agent from being rewarded for closing while it is itself being pursued, and the second one attenuates the reward below the distance D_c, which discourages the agent from driving into an overshoot. The parameter D_c therefore acts as a soft no-overshoot boundary at which the agent is expected to have established a firing position rather than to continue to reduce the distance.

The two aiming terms represent the firing opportunity of the two aircraft. Because the damage inflicted by the gun enters the return only through the outcome of the engagement, these terms provide the dense signal that guides the nose of the aircraft onto the firing line. The aiming reward r_{gb} is active when the track angle falls below the alignment threshold $\hat{\theta}_g$, so that it rewards a settled gun solution rather than a mere pointing of the aircraft towards the opponent, and it is weighted by $\Gamma_B(D)$ so that the reward is concentrated in the range band where such a solution is sustainable. The opponent aiming penalty r_{gr} mirrors this construction for the adversary: it becomes active when the adverse angle exceeds the threshold $\hat{\theta}_r$, that is, when the opponent's nose is aligned with the ego aircraft, and it is weighted by $\Gamma_R(D)$, so that it imposes an increasing cost on conceding the offensive position inside the opponent's firing envelope.

Finally, the low-altitude penalty r_{deck} acts as a flight-safety constraint. It is negligible while the aircraft remains above h_{deck} and grows rapidly below that altitude, so that it prevents the policy from trading an excessive amount of altitude for speed or from entering the ground-proximity regime during aggressive manoeuvring. The progressive design of the two stages, together with the terminal terms given below, enables the agent to develop tactically meaningful offensive and defensive behaviour. The structure of the stage-2 reward follows the approach adopted in the ADT project [2].

Besides the per-step terms of the two stages, the outcome of the engagement is accounted for by the terminal terms listed in Table 5, which are applied once when an episode terminates.

In Table 5, t_f denotes the terminal time of the episode, τ_{rem} the number of decision steps remaining before the maximum episode length is reached, and s_{kill} and s_{loss} are indicators that equal one when the opponent and the ego aircraft are destroyed in the air, respectively. The health margin is evaluated whenever neither aircraft has crashed, and it converts the residual health of the two aircraft into a continuous

Table 5 Terminal reward terms, applied when an episode terminates.

Terminal term	Mathematical expression
Health margin	$r_{hp} = w_{hp}\,(\mathrm{HP}(t_f) - \mathrm{HP}_{oppo}(t_f))$
Kill bonus	$r_{kill} = s_{kill}\,w_{kill}\,\tau_{rem}$
Loss penalty	$r_{loss} = -\,s_{loss}\,w_{loss}\,\tau_{rem}$

outcome signal, so that an engagement which reaches the time limit is not scored as a neutral event but reflects the damage exchanged during it. The kill bonus rewards the destruction of the opponent, and the loss penalty penalizes the destruction of the ego aircraft. Since both are proportional to the remaining time, an engagement that is decided early is rewarded, or penalized, more strongly than one that is decided late, which introduces the time pressure that is characteristic of a gun engagement: a firing solution that is obtained early leaves the opponent with less opportunity to reverse the situation, whereas a delayed solution consumes the time budget available for the engagement.

3.2.2 Reward Composition and Parameter Calibration

The parameters of the reward functions given in Tables 3–5 fall into two groups, which are determined in different ways. The first group consists of the thresholds and scaling factors that describe the physical setting: the boundaries of the gun envelope in Eq. (9), the alignment thresholds and the range bands of the two aiming terms, the reference condition of trimmed flight, and the safety altitude. These quantities characterize the weapon, the flight envelope of the aircraft and the safety requirements of the manoeuvring airspace, and they are therefore fixed by the engagement scenario rather than being free parameters of the learning problem. The second group consists of the weights $w_\bullet$ and of the slopes $k_\bullet$ of the shaping terms. Since the terms interact, and since their numerical values are meaningful only relative to one another, these coefficients cannot be derived in closed form; they were calibrated iteratively, as described at the end of this subsection.

For the first stage, the reference values are $\delta_a^* = 0$, $\phi^* = 0$ and $\delta_T^* = 0.4$, the last of which is the throttle setting prescribed at the initial condition of every episode. The scaling factors are $s_{sa} = s_{sr} = 0.5$, so that a half-deflection aileron command or a bank angle of 0.5 rad reduces the corresponding term to half of its maximum, and $s_{st} = 0.1$, which concentrates the throttle term around the cruise setting and therefore penalizes large throttle excursions strongly. The weights are $w_{st} = 8$ and $w_{sa} = w_{sr} = 4$. The windows of the two potential-difference terms are $L_{ap} = 10$ and $L_{alt} = 20$ decision steps, that is, 1 s and 2 s of engagement time; a substantially shorter window would react to the numerical noise of the actuator commands, whereas a longer one would delay the response to genuine changes of the flight condition. Their weights are $w_{ap} = 4$ and $w_{alt} = 8 \times 10^4$, the latter compensating for the small normalized magnitude of the altitude increments, as discussed above.

For the second stage, the transition between the offensive and the defensive regime is centred at $\hat{\theta}_a = 0.5$, that is, $\theta_{adv} = 90°$, with slope $k_a = 18$, which spreads the transition over approximately 40° and thereby avoids a discontinuous interpretation of the relative geometry. The closure rate is normalized by $V_{ref} = 500$ ft/s, a value representative of the closing speeds encountered by the modelled aircraft in within-visual-range engagements, so that a typical approach produces a term of order unity; the no-overshoot attenuation is centred at $D_c = 2900$ ft with slope $k_D = 1/500$ ft^{-1}, so that the term is essentially removed inside the range where a tracking solution should already have been established. The alignment thresholds follow the geometry of the weapon: the aiming reward uses $\hat{\theta}_g = 1/180$ together with $k_g = 10^5$, so that it is active only when the nose is aligned with the firing line to within about 1°, whereas the opponent aiming penalty uses $\hat{\theta}_r = 178/180$ with $k_r = 800$, so that it grows as soon as the opponent's nose approaches the ego aircraft, with a gentler slope that provides a usable gradient before the opponent's firing solution is complete. The range weighting functions of Eq. (15) use $\beta_B = \beta_R = 2.5$, $D_{B,1} = 1000$ ft, $D_{B,2} = 2900$ ft, $D_{R,1} = 400$ ft and $D_{R,2} = 4100$ ft, with slopes $k_{B,1} = k_{B,2} = 1/50$ ft^{-1}, $k_{R,1} = 1/35$ ft^{-1} and $k_{R,2} = 1/200$ ft^{-1}. The bands of the two functions thus occupy the middle of the corresponding firing ranges and retain a margin at both ends, which reflects the fact that a firing solution is transient at very short range and uncertain near the far boundary. The weight of the relative position term is $w_{rp} = 1$, so that this term defines the unit of the tactical shaping scale, whereas the low-altitude penalty uses $w_{deck} = 4$, that is, four times that unit, together with $h_{deck} = 1300$ ft and $k_h = 1/20$ ft^{-1}. The larger weight is deliberate, because entering this regime endangers the aircraft and therefore dominates any tactical consideration, while the threshold lies well below the altitude band in which the engagements are initialized (10000 ± 1000 ft), so that the term remains inactive during normal manoeuvring. The terminal terms use $w_{hp} = 10$, $w_{kill} = 3.7$ and $w_{loss} = 1$. With $\mathrm{HP}_0 = 3$, the health margin spans ± 30, whereas the kill bonus reaches approximately 2200 when the opponent is destroyed at the beginning of an episode. The outcome of the engagement therefore dominates the accumulated shaping reward, which is essential in the present setting: because the gun inflicts damage continuously and the result of an engagement is decided by the residual health, a return dominated by per-step terms could be accumulated without ever resolving the engagement, a behaviour that the high proportion of drawn episodes reported in Section 4.3.1 would otherwise encourage.

Because these coefficients interact, they were calibrated iteratively rather than optimized, by adjusting a small number of them at a time and inspecting the resulting behaviour both in the training curves and in the statistics of the evaluation episodes. The diagnostics used for this purpose are the per-term reward averages, which are recorded separately for every component during evaluation, together with the outcome statistics of the engagement, namely the win, draw and loss rates, the proportion of the engagement time during which the opponent remains inside the WEZ, the mean track angle, and the crash rate. The following observations determined the direction of the adjustments. When the recorded trajectories exhibited sustained oscillation of the control surfaces or an unsteady altitude trace, the weights of the corresponding rate terms, w_{ap} and w_{alt}, were increased until the oscillations disappeared; conversely,

when the agent became reluctant to manoeuvre and the engagements degenerated into long-range circling, these weights were reduced, because an excessive smoothness penalty suppresses the aggressive manoeuvring that the tactical terms are intended to encourage. When the policy retained a residual bank angle or an off-trim throttle setting, the weights of the corresponding asymptotic terms, w_{sr}, w_{sa} and w_{st}, were increased. In the second stage, when the agent learned to point its nose at the opponent but failed to reduce the distance, the closure term was strengthened by increasing w_{cr}; when the recorded trajectories showed repeated overshoots at close range, that is, a rapid reversal of the closure rate after the distance had decreased below the gun range, D_c was moved closer to the effective firing band and the weight of the aiming reward was increased, so that a settled firing solution was rewarded rather than the mere approach. When the agent spent a substantial part of the engagement in front of the opponent without attempting to disengage, the opponent aiming penalty was extended to longer ranges by increasing $D_{R,2}$ and its amplitude β_R was increased, so that the cost of conceding the offensive position became comparable to the benefit provided by the angular terms. When the proportion of crashed episodes was not negligible, h_{deck} was raised until the aircraft stopped entering the low-altitude regime. Finally, when the shaping terms indicated that the agent had learned to hold the offensive position while the proportion of drawn engagements remained high, w_{kill} and w_{hp} were increased, so that deciding the engagement replaced the accumulation of shaping reward as the dominant objective of the policy.

Each adjustment was validated by retraining the policy and re-evaluating it against a fixed set of opponents; an adjustment was retained only when it improved the outcome statistics without increasing the crash rate or degrading the quality of the flight, and it was reverted otherwise. The values reported above are those retained by this procedure, and they are listed together with the remaining experimental settings in Section 4. It should be noted that these values reflect the balance between the terms discussed in this section rather than a unique optimum, and they are reported in full so that the reward structure can be reproduced.

3.2.3 Opponent Curriculum

Objective curriculum cannot regulate the game difficulty during training. If the agent combats with an aggressive opponent from the very beginning, it may be shot down frequently before learning basic evasive maneuvers. As a result, the sparse reward makes it difficult for the policy network to extract useful gradients from a large number of failed episodes and potentially causing convergence to random behavior. To address this issue, we propose a dynamic-threshold opponent curriculum. By adjusting the opponent's target, the game difficulty increases smoothly.

Opponent curriculum is achieved by changing how the opponent selects its target. Figure 4 shows a low-threat opponent and a high-threat opponent. Specifically, we define a threshold value $p(t) \in [0, 1]$ that decays linearly with the training step t:

$$p(t) = \max\Big(0,\ 1 - \frac{t}{T}\Big), \tag{16}$$



Fig. 4 Low-threat opponent and high-threat opponent in the opponent curriculum.

where T denotes the total number of steps over which the curriculum decays. During the initialization of each training episode, the system samples a random variable ξ uniformly from $(0, 1)$ and assigns a target point to the opponent according to the comparison between ξ and the current threshold $p(t)$:

- If $\xi \leq p(t)$, the opponent is treated as a low-threat opponent. The system generates a target point randomly within the airspace, and the opponent follows a policy that flies toward this target point. In this mode, the opponent does not attack the ego and poses only a minimal threat.
- If $\xi > p(t)$, the opponent is treated as a high-threat opponent. Its target point is updated in real time to the current position of the ego. In this case, the opponent behaves aggressively and continuously attempts to obtain gun-firing opportunities, imposing substantially greater adversarial pressure on the ego.

As training proceeds, $p(t)$ decreases linearly from 1 to 0, so the probability of encountering a high-threat opponent increases correspondingly. At the beginning of training, the agent faces almost exclusively harmless opponents flying toward random points, which allows it to safely explore the flight envelope and accumulate basic flight experience. As the training time increases, high-threat opponents begin to appear with increasing frequency, forcing the policy to gradually learn defensive, evasive, and counter-engagement behaviors. This difficulty scheduling design enables a smooth transition from non-adversarial flight to adversarial dogfighting. It helps alleviate the instability caused by introducing strong opponents too early, while also reducing the risk of policy stagnation induced by prolonged exposure to weak opponents.

3.3 Dynamic Opponent Pool

The opponent curriculum described above regulates the training difficulty by changing the opponent's target assignment. However, this mechanism only changes the opponent's behavioral objective during an episode, while the opponent policy itself, i.e., its model parameters and intrinsic combat capability, remains unchanged. Therefore, although opponent curriculum can provide a smooth difficulty transition in the early stage of training, it is insufficient to continuously improve the adversarial competence of the opponent.

To further enhance training, the ego policy should be exposed to opponents with progressively stronger and more diverse combat abilities. Training against a fixed opponent often causes the policy to overfit to a particular behavior pattern, whereas pure self-play may induce policy oscillation due to cyclic dominance. To address this issue, this study proposes a dynamic opponent pool. Fixed benchmark opponents are

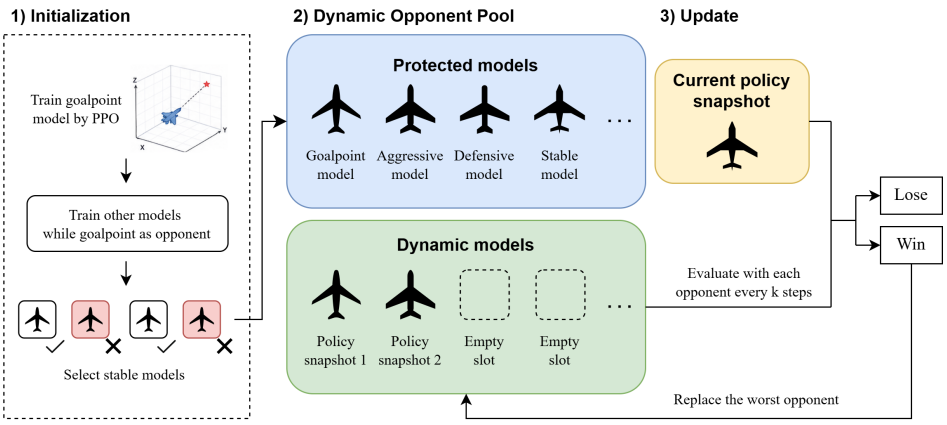


Fig. 5 Dynamic opponent pool.

combined with competitive historical snapshots to provide the policy with a continuously evolving and behaviorally diverse set of adversaries. Details are shown in Figure 5.

3.3.1 Opponent Pool Initialization

The initialization of the opponent pool is critical for stable policy learning. If the initial opponents are overly simple, the agent may overfit to limited behavior patterns, and when stronger opponents are introduced later, the learned policy and value estimates may transfer poorly, leading to performance degradation. Conversely, if the initial opponents are too difficult, the agent may fail to obtain sufficient meaningful interactions. To alleviate this cold-start problem, this paper introduces a goalpoint navigation policy as an initial protected opponent.

The goalpoint model is defined as follows: from an arbitrary initial state, the aircraft will fly toward a randomly generated target point in space. Its observation consists of the ego aircraft state together with the geometric relationship between the aircraft and the target point. Instead of using the absolute coordinates of the target point, the model takes relative features as input, including relative azimuth, elevation, and distance. This design prevents the policy from memorizing specific spatial locations.

The necessity of this design becomes clear when the way in which opponent diversity is obtained is considered. In the conventional approach, every opponent exhibiting a distinct maneuvering style has to be specified by its own reward function and produced by its own separate training run, so that enlarging the opponent set costs one additional reward design and one additional training run per style, and the resulting policies remain unrelated to one another. The goalpoint model removes this dependence, because the target point enters the policy as a conditional input rather than as part of its learning objective. Changing the way in which the target point is generated therefore changes the maneuvering habit of the same policy, without any modification of the reward function and without any retraining. For example, a target point

drawn at random in the airspace yields a non-adversarial opponent that simply cruises, climbs or descends toward an arbitrary location; a target point that is overwritten in real time with the current position of the ego aircraft turns the same policy into an aggressive pursuer that keeps its nose on the ego aircraft and closes the range; and a target point that moves along a straight line, along a randomly perturbed path, or along a helical trajectory drives the same policy into sustained banked turns, weaving and altitude changes while it tracks the point, which are otherwise obtained only from separately trained evasive policies. A single set of parameters thus covers a family of maneuvering behaviors, and switching between them is a change of one input variable rather than a new reward design followed by a new training run. This is the significance of the goalpoint model: it converts opponent diversity, which would otherwise have to be paid for by repeated reward engineering and repeated training, into a cheap re-parameterization of the target point.

To facilitate the learning of this navigation task, a composite reward function is designed. Four general reward terms are used, namely an aileron magnitude penalty, an action smoothness penalty, a low-altitude penalty, and a closure-rate error term. The first two and the low-altitude penalty are of the same form as the corresponding terms of Tables 3 and 4, whereas the closure-rate error term is specific to this auxiliary task: it is an asymptotic error potential whose target is a prescribed closure rate $V_c^* = 700$ ft/s, so that the aircraft approaches the target point at a moderate rate instead of accelerating towards it. The parameters of these four terms are $w_{ail} = 1.0$ for the aileron magnitude penalty, $w_{ap} = 3.5$ with $L_{ap} = 10$ for the action smoothness penalty, $w_{deck} = 4$, $h_{deck} = 1300$ ft and $k_h = 1/20$ ft^{-1} for the low-altitude penalty, and $V_c^* = 700$ ft/s with scaling factor $s = 800$ ft/s and unit weight for the closure-rate error term. In addition to these terms, the reward function incorporates two task-specific components to actively guide the aircraft: a distance reward and a heading reward. Both components are computed using a scaled asymptotic error form,

$$r_i = w_i \left(1 - \frac{|x_i - x_i^*|/s_i}{|x_i - x_i^*|/s_i + 1} \right) = w_i \frac{s_i}{|x_i - x_i^*| + s_i}, \tag{17}$$

where x_i is the current value of the corresponding variable, x_i^* is the target value, s_i is the scaling factor, and w_i is the weight of the component. The form of Eq. (17) is identical to that of the first three terms of Table 3, so that the same interpretation applies: each component equals its weight w_i when the corresponding error vanishes, retains half of that value when the error equals s_i, and decays smoothly for larger errors without introducing a singularity.

For the distance reward, the target distance is zero, i.e., $x_i^* = 0$. Let $d(t)$ denote the relative distance between the aircraft and the target point in feet. Following Eq. (17), the distance reward is defined as

$$r_{\text{dist}}(t) = w_{\text{dist}} \frac{s_{\text{dist}}}{d(t) + s_{\text{dist}}}, \tag{18}$$

where $w_{\text{dist}} = 4.0$ and $s_{\text{dist}} = 2000$ ft. The scaling factor is comparable to the separation at which the target points are generated, which are placed several thousand feet

away from the aircraft in the horizontal plane and between 8000 and 20000 ft in altitude, so that the term retains an informative gradient over the whole approach and decays smoothly as the target point is reached; the large weight makes this term the dominant component of the reward of the auxiliary task, so that reaching the target point, rather than refining the flight condition, is its primary objective.

For the heading reward, since the desired heading error is zero, the reward is likewise given by

$$r_{\text{head}}(t) = w_{\text{head}} \frac{s_{\text{head}}}{|\theta_{tra}(t)| + s_{\text{head}}}, \tag{19}$$

where $\theta_{tra}(t)$ is the angle between the nose of the aircraft and the line of sight to the target point, and the parameters are $w_{\text{head}} = 0.2$ and $s_{\text{head}} = 0.2$ rad, that is, approximately 11.5°. The scaling factor is comparable to the angular offset that the aircraft must remove in order to fly towards a target point, and the resulting term decreases from its maximum of 1 when the aircraft points directly at the target point to approximately 0.1 when the target point lies abeam, so that it acts as a persistent pointing signal which refines the direction of the flight without competing with the distance term, whose weight is twenty times larger. The target point is considered to be reached when the distance falls below 500 ft, at which point the episode terminates.

In this work, the goalpoint model is pretrained separately using PPO, and the target point is randomly reset at every episode. The resulting goalpoint policy can guide the aircraft toward arbitrary target locations in the airspace using smooth control commands. Using this pretrained goalpoint model as the opponent, a batch of preliminary combat-capable models is trained by applying the reward function detailed in Table 4. These policies are then evaluated according to their training curves, flight safety, and behavioral characteristics. Models with reliable training performance, low crash frequency, clear offensive or evasive behaviors, and diverse maneuvering styles are selected to form the initial protected opponent set. In this way, the initialized opponent pool contains not only the original goalpoint policy, but also several derived opponents that already possess basic maneuvering and combat interaction capability. Compared with initializing the pool using only handcrafted opponents, this procedure provides a broader opponent distribution. Furthermore, because the navigation targets of all opponents within the pool can be flexibly overridden, the system can seamlessly implement the opponent curriculum by toggling opponent behaviors between low threat and high threat. Combined with the objective curriculum, this integrated pipeline systematically guides the agent from basic flight control to complex air combat mastery.

3.3.2 Opponent Pool Structure and Update

Specifically, the proposed framework maintains an opponent pool with fixed capacity. At the beginning of each training episode, an opponent is independently sampled from the pool for each parallel environment. The opponent pool consists of two components. The first is a set of protected opponents initialized using the procedure described in the previous subsection, which provide stable baselines. The second is a set of dynamically updated policy slots used to store historically strong versions of the policy during training. The protected opponents cover representative behavior patterns with

different levels of complexity and tactical styles. They remain unchanged throughout training so as to prevent the performance of the opponent pool from degrading due to frequent replacement. In contrast, the dynamic slots are filled progressively as training proceeds and are updated through periodic evaluation.

Let the evaluation interval be denoted by K. Whenever the number of training steps reaches an integer multiple of K, the current policy is frozen to form an evaluation snapshot, which is then matched against every opponent in the pool for N independent episodes. To avoid interference from reward shaping, the update criterion does not rely directly on training return. Instead, it is constructed only from game outcomes and the remaining health at the end of engagement. For opponent i, the corresponding score is defined as

$$S_i = w_i - l_i + 0 \cdot d_i + 0.5 f_i + 0.01\big(hp_i - hp_i^{oppo}\big), \tag{20}$$

where w_i, l_i, and d_i denote the win rate, loss rate, and draw rate, respectively; f_i denotes the opponent self-crash rate; and hp_i and hp_i^{oppo} denote the average remaining health percentages of the ego and the opponent aircraft, respectively. In this way, it offers a more direct measure of actual dogfight effectiveness and reduces the influence of reward hacking.

The update rule is as follows. If the opponent pool still contains empty dynamic slots, the current policy snapshot is inserted directly into one of them. If all dynamic slots are already occupied, the current policy is compared with the replaceable opponents in the pool, and it replaces the weakest one if its performance is superior. The protected opponents are never removed.

By preserving fixed benchmark opponents together with a limited number of high-quality historical snapshots, the dynamic opponent pool helps balance adversarial diversity, training stability, and computational cost. Moreover, the design is compatible with the parallel sampling framework: different sub-environments independently sample opponents at each episode, so that a single batch of experience contains multiple adversarial styles. This design alleviates the bias caused by a single-opponent distribution and improves the generalization of the policy.

In summary, the baseline is trained solely against non-reactive, level-flight opponents that cannot perceive the ego aircraft, and it operates without curriculum learning or a dynamic opponent pool. To achieve better combat performance, our proposed method first constructs an initial opponent pool using the goalpoint model to alleviate the cold-start problem. Subsequently, the policy is trained by integrating the curriculum learning alongside the dynamic opponent pool to ensure stable learning and robust generalization. The performance of this method will be systematically compared against the baseline in Section 4.

4 Experiments and Evaluation

4.1 Experimental Setup

This section describes the training framework and hardware platform in the experiments. All experiments are implemented based on Stable-Baselines3 and are trained

using PPO. Except for the curriculum learning and the opponent selection, all other training configurations are kept identical across different methods to ensure a fair comparison.

The experimental comparison is organized from two perspectives. First, to evaluate the contribution of each proposed component, we start from a baseline and progressively consider the use of curriculum learning and the dynamic opponent pool. This yields four MLP-based models: the baseline model without curriculum learning or dynamic opponent pool (Baseline), the model with the dynamic opponent pool only (BD), the model with curriculum learning only (BC), and the model using both components (BDC-MLP). Second, to study the influence of the policy architecture, we further compare two implementations of the complete BDC method, namely an MLP-based version (BDC-MLP) and an LSTM-based [30] version (BDC-LSTM).

For the component ablation experiments, the MLP-based policy architecture is used by default to ensure that the compared models differ only in the use of curriculum learning and the dynamic opponent pool. For the MLP-based policy, the policy and value networks are parameterized by multilayer perceptrons and share a common feature extractor. For the LSTM-based policy, recurrent layers are further introduced to capture temporal dependencies in the observation sequence. The detailed hyperparameter settings for both architectures are summarized in Tables A3 and A4.

Experiments were conducted on a workstation running Ubuntu 24.04.2 LTS, equipped with an Intel® Core™ i9-14900KF CPU and a NVIDIA GeForce RTX™ 5070 Ti GPU.

4.2 Curriculum Learning Ablation

To evaluate the effect of the curriculum learning on training stability and convergence performance, an ablation study is conducted. The experiments record the evolution of the mean evaluate return of three methods over 3.0×10^8 environment interaction steps. The corresponding training curves are shown in Figure 6.

From the overall trend, the BDC method outperforms the baseline. Although the baseline exhibits some improvement at the early stage of training, it begins to plateau after approximately 5×10^7 steps, with the mean return remaining around 500 for a long period. BDC method converges to approximately the 1000–1250 range, suggesting that curriculum learning and dynamic opponent pool helps improve policy performance.

A more direct view of the role of curriculum learning in training stability can be obtained by comparing the BDC method with BD. After removing curriculum learning, the model still benefits from the dynamic opponent pool, but the training curve shows pronounced fluctuations and experiences clear performance collapses around 4×10^7 and 1.3×10^8 steps. During the early stages of training, the agent faces three challenges: complex flight control, air-combat objectives, and an evolving opponent. Forcing the agent to address these simultaneously renders the optimization problem difficult. Ultimately, this may lead to unstable policy updates or even catastrophic forgetting.

By contrast, the BDC method gradually increases task difficulty through the curriculum. This process significantly alleviates the exploration burden at the beginning

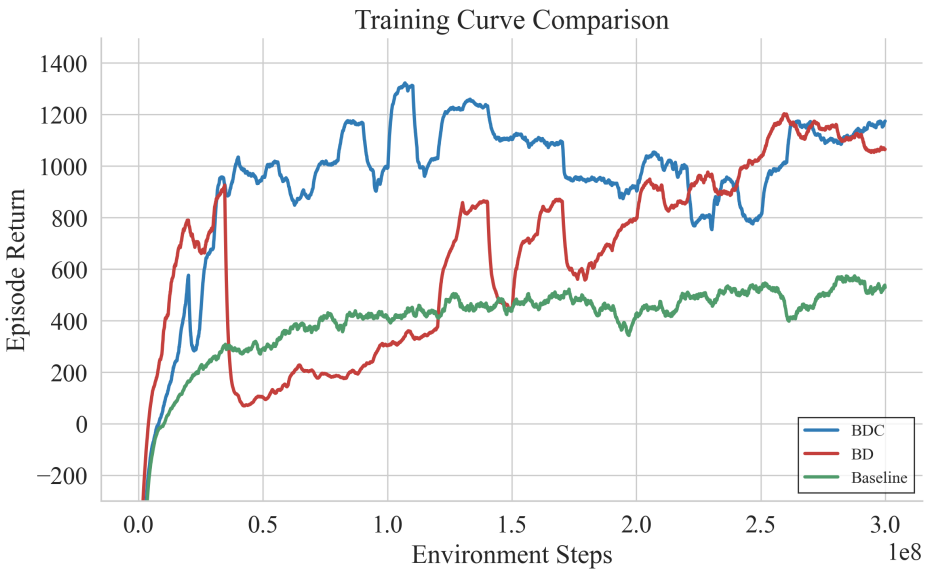


Fig. 6 Episode return of the Baseline, BD, and BDC(MLP) methods. "Baseline" denotes standard PPO with a fixed opponent pool; "D" and "C" represent the Dynamic opponent pool and Curriculum learning components, respectively.

of training, allowing the model to achieve higher final returns while avoiding large performance oscillations. Although some fluctuations still remain in the later training stage as the dynamic opponent pool continues to evolve, the overall variance is clearly lower than BD method. These results demonstrate that the curriculum learning enhances the stability and robustness of the training process.

4.3 Evaluation

4.3.1 Benchmark Pool Evaluation

To evaluate the generalization ability and tactical performance of the models, we construct a standardized benchmark opponent pool of eight policies. These opponents are selected snapshots obtained from various training iterations and configurations. Their behavioural differences are induced by the reward terms on which they were trained, since the initial engagement geometry is randomized in the same way for all of them; the pool therefore spans a range of flight behaviour rather than a range of engagement geometries. Because a qualitative description of eight policies is difficult to verify, every opponent is characterized by direct measurement as set out below and summarized in Table 6. Representative engagement scenarios of the eight opponents are illustrated in Figure 7, where the red aircraft denotes the benchmark opponent.

To describe the behaviour of the benchmark opponents quantitatively, each of the eight policies was flown for 60 s from an identical initial condition and its own control commands, attitude and trajectory were recorded at every decision step. The opposing aircraft is a scripted constant-command drone that is identical for all eight policies, so

the only quantity that differs between the episodes is the behaviour of the opponent being measured. No reward is used in this measurement, and no engagement outcome is scored; the reported quantities therefore describe how a policy flies rather than how well it performs. Table 6 reports five of them. The mean bank angle $|\phi|$ measures how steeply a policy is willing to bank. The mean climb rate separates a policy that gains altitude from one that loses it. The mean elevator command distinguishes a policy that holds a sustained pull from one that remains close to trim. The mean speed indicates the airspeed the policy settles at, and the mean closure rate towards the other aircraft is positive when a policy reduces the range and negative when it opens it.

Five behaviour classes emerge from these measurements, and their measured signatures are as follows.

- **Class I (opponent 1), sustained maximum-manoeuvre flight.** This is the only policy that flies at full throttle throughout, and its mean elevator command saturates at -1.000, that is, it holds a full nose-up command for the whole engagement. It banks steeply (86.3° on average, and more than 60° for 66% of the time), turns fastest of all eight policies, and climbs continuously at +113.8 ft/s, reaching the highest mean speed and the highest specific energy of the pool.
- **Class II (opponent 2), high-manoeuvre energy trade.** This policy manoeuvres nearly as hard as opponent 1 (60.1° mean bank, 48% of the time above 60°) but throttles back to an idle setting, and it shows the highest control activity and the highest roll-reversal rate of the pool. Its mean speed and its specific energy are the lowest of the eight, which identifies it as a policy that trades energy for angular advantage.
- **Class III (opponent 3), moderate manoeuvre.** Bank angle (47.8°), turn rate and elevator command (-0.236) all lie between those of classes I–II and those of class V, and the policy also throttles back. It is best described as an intermediate case between sustained manoeuvring and level flight.
- **Class IV (opponent 4), steady descent.** This is the only policy whose altitude falls throughout: it descends at -374.4 ft/s on average, climbs during 1% of the episode, and never exceeds a mean bank of 8.1° or an elevator command of +0.033, that is, it neither turns nor pulls. Its airspeed is high (847 ft/s) because it accelerates in the descent, and the episodes recorded for this opponent are correspondingly shorter than those of the other seven.
- **Class V (opponents 5–8), near-level cruise flight.** These four policies hold a small bank angle (3.2° to 24.2°), keep the elevator command within 0.17 of trim, and reverse their roll essentially never; their speed range spreads more widely (444 to 700 ft/s) than their manoeuvring does. Opponent 6 is the most manoeuvrable member of this class, which is why its bank angle is reported separately in the table.

Two properties of the pool follow directly from these numbers. First, the pool is strongly asymmetric in manoeuvre intensity. Opponents 1–3 fly with a mean bank between 47.8° and 86.3° and a sustained pull on the elevator, whereas opponents 5–8 remain close to level flight with a mean bank below 25° and an elevator command within 0.17 of trim. Any evaluation that reports a single aggregate over the eight opponents therefore mixes two very different regimes, and the win rates reported below

are weighted accordingly by the composition of the pool. Second, none of the eight opponents directs its manoeuvring at the other aircraft in this measurement. The mean closure rate is negative for all eight policies, and the fraction of a policy's turning that is explained by the line-of-sight angle to the other aircraft does not exceed $R^2 = 0.11$ for any of them, with five of the eight below 0.02. This is consistent with the reward functions on which the opponents were trained: opponents 5–8 are level-flight policies whose reward contains only flight-quality terms, namely control smoothing, altitude containment and roll suppression, and no term that depends on the relative geometry, so target-directed behaviour is not expected of them. The benchmark pool is therefore best read as a ladder of flight behaviour rather than of adversarial pressure, and it should not be assumed that all eight opponents actively contest the engagement.

Table 6 Measured flight behaviour of the eight benchmark opponents, grouped into five behaviour classes. Every policy was flown for 60 s from an identical initial condition with its own control commands and trajectory recorded; the opposing aircraft is a scripted constant-command drone common to all eight policies, and no reward or engagement outcome is used. Units: mean bank angle $|\phi|$ in degrees, climb rate in ft/s, elevator command normalised to $[-1, 1]$ with a negative command pitching the nose up, speed in ft/s and closure rate in ft/s.

Class	Opp.	Mean $\lvert\phi\rvert$	Climb rate	Elevator cmd	Speed	Closure to opp.
I	1	86.3	+113.8	−1.000	888	−155
II	2	60.1	+6.4	−0.618	475	−394
III	3	47.8	+36.0	−0.236	572	−297
IV	4	8.1	−374.4	+0.033	847	−102
V	5	5.1	+39.0	−0.066	566	−358
	6	24.2	−7.7	−0.078	700	−549
	7	10.2	+66.6	−0.105	509	−341
	8	3.2	+33.2	−0.165	444	−500

Five models are compared: the baseline model (Baseline), a model with curriculum learning only (BC), a model with the dynamic opponent pool only (BD), and two versions of the BDC method, namely the multilayer perceptron version (BDC-MLP) and the long short-term memory version (BDC-LSTM). During evaluation, each model is matched against all eight opponents, with 1000 independent Monte Carlo simulations conducted for each matchup. In these simulations, the Monte Carlo sampling is specifically applied to the initial states of the aircraft, ensuring that both the starting position and attitude are randomly initialized for every episode to thoroughly assess the model's robustness.

Figure 8 shows the win/draw/loss distributions on the benchmark opponent pool. The Baseline is almost unable to secure victories, achieving a win rate of only 0.42%, with the vast majority of engagements ending in draws. This indicates that training only against fixed simple opponents may lead to policy overfitting. After introducing curriculum learning and the dynamic opponent pool separately, the win rates increase

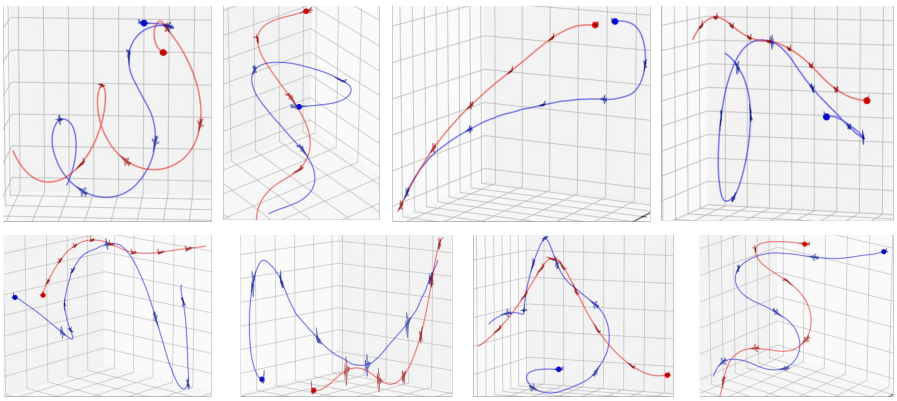


Fig. 7 Representative engagement scenarios of the eight benchmark opponents (the red aircraft).

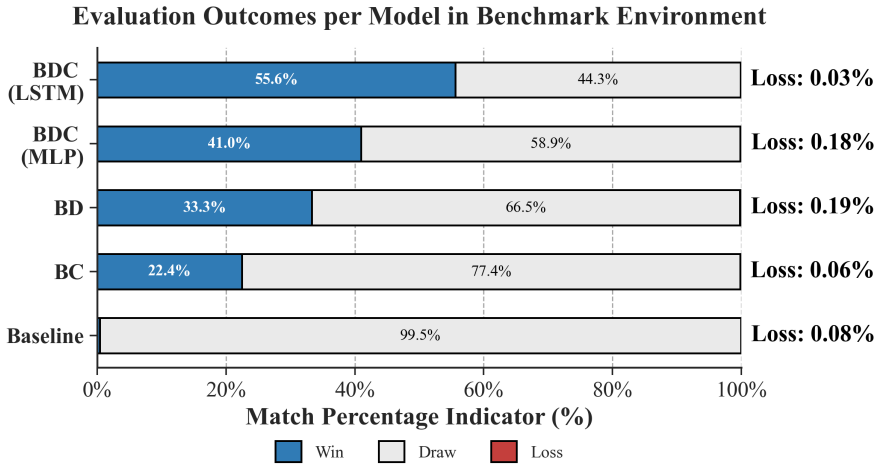


Fig. 8 Win, draw, and loss rates of different models against the benchmark opponent pool over 8×1000 simulated engagements per model.

to 22.4% and 33.3%, respectively, showing that both components can independently improve generalization performance. The BDC method further improves the overall adversarial performance, with BDC-MLP and BDC-LSTM achieving win rates of 41.0% and 55.6%, respectively. The BDC-LSTM model therefore outperforms the baseline by approximately 55.2 percentage points.

Table 7 and Figure 9 report the energy margin and line-of-sight figure R^2 for the five models. The energy margin ΔE_s is the mean difference between the specific energies of the two aircraft, $E_s = h + V_T^2/(2g)$, where h is the altitude, V_T the true airspeed and g the gravitational acceleration. Holding a higher specific energy than the opponent is the objective of the energy-fight tactic [31, 32], because that energy can later be converted into a firing position, whereas the reward compares the two aircraft only through their angles and their range. The line-of-sight figure R^2 is the

Table 7 Capability metrics of the five compared models, measured with the protocol of Section 4.3.1 (8×150 engagements per model with randomised initial states). ΔE_s is the mean specific-energy margin over the opponent, and the line-of-sight figure R^2 is the squared correlation between the line-of-sight angle to the opponent and the signed turn rate, the sign being taken from the bank angle so that the heading changes produced by rolling do not contaminate the estimator. Neither quantity appears in any term of the reward function, so the comparison is not circular.

Model	ΔE_s (ft)	LOS R^2
Baseline	-2941	0.24
BC	+1526	0.22
BD	+1815	0.24
BDC-MLP	+3758	0.35
BDC-LSTM	+4679	0.36

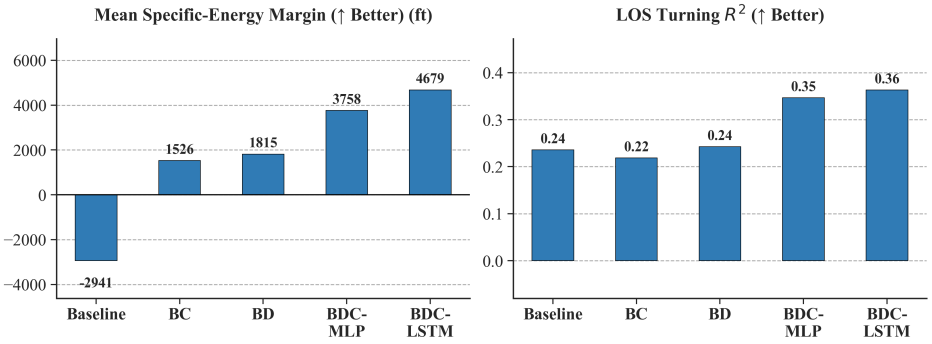


Fig. 9 Comparison of the reward-independent capability metrics of the five models: mean specific-energy margin over the opponent (left) and line-of-sight turning figure R^2 (right), evaluated over the same engagements as Table 7.

squared correlation between the line-of-sight angle to the opponent and the signed turn rate, the sign being taken from the bank angle so that the heading changes caused by rolling do not contaminate the estimator. It is the fraction of the policy's turning that is explained by the position of the adversary, and it therefore separates a policy that steers by the opponent from one that follows a trajectory of its own; it cannot be optimized directly, because the reward contains no term in the turn rate.

Both quantities separate the models in the same way as the outcome statistics. The energy margin is the clearest of them: it increases monotonically with the win rate, from -2941 ft for the baseline to $+4679$ ft for BDC-LSTM, and BDC-LSTM is the only model that finishes all eight matchups with a positive margin, its smallest value being $+2955$ ft, whereas the baseline finishes six of the eight with a negative margin. The line-of-sight figure distinguishes the two variants of the proposed method from the other three models: BDC-MLP and BDC-LSTM reach $R^2 = 0.35$ and 0.36, against 0.22 to 0.24 for the baseline, BC and BD. A larger part of their turning is therefore accounted for by the position of the adversary, which is the behaviour expected of a

policy that manoeuvres in response to the opponent instead of following a trajectory of its own.

4.3.2 Cross-Play Evaluation

In addition to the benchmark opponent pool evaluation, we further conduct cross-play experiments to compare the relative combat performance of different models. The models are evaluated in a round-robin manner, with each model competing against every other model. We evaluated the win rate of each model and the difference in remaining HP between the ego and the opponent at the end of each engagement. The results are shown in Figure 10.

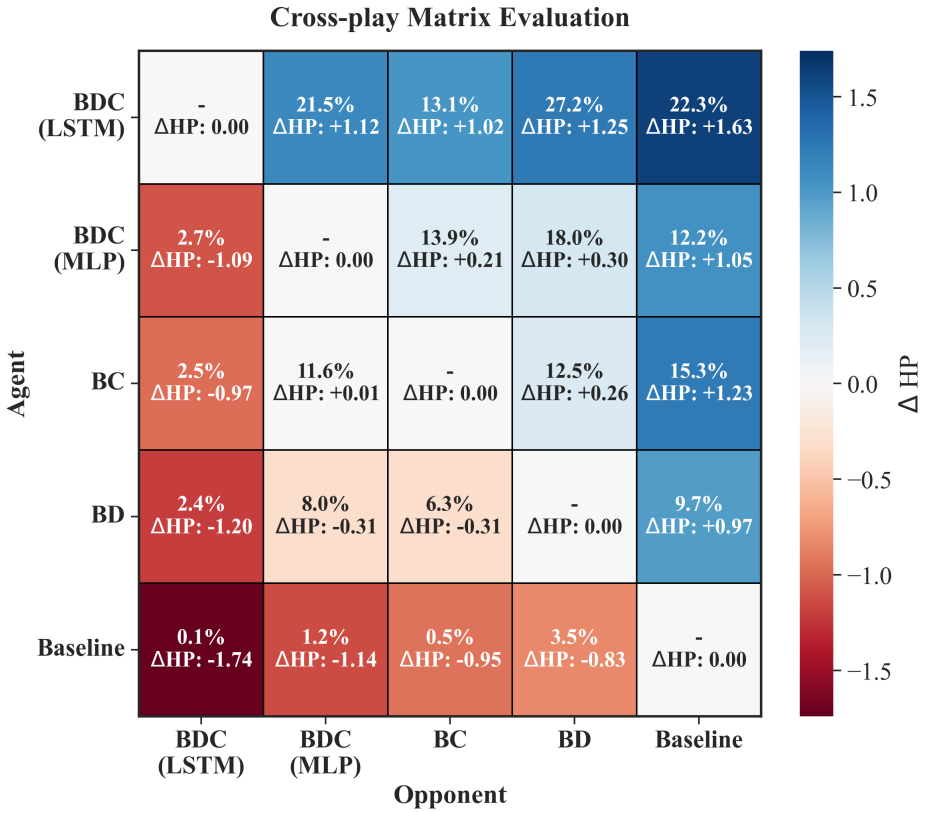


Fig. 10 Cross-play results among the ablation models. Each cell reports the win rate and the remaining-HP difference (ΔHP) of the row agent against the column agent.

In Figure 10, the color indicates the HP difference between the row model and the column model. The results show that models using BDC method achieve higher win rates and positive ΔHP values against all ablation models, whereas the baseline is consistently at a disadvantage when facing the other methods. Introducing curriculum

learning alone or the dynamic opponent pool alone can both improve policy quality, but neither matches the BDC method. Furthermore, a comparison across network architectures reveals that LSTM-based policies outperform their MLP counterparts.

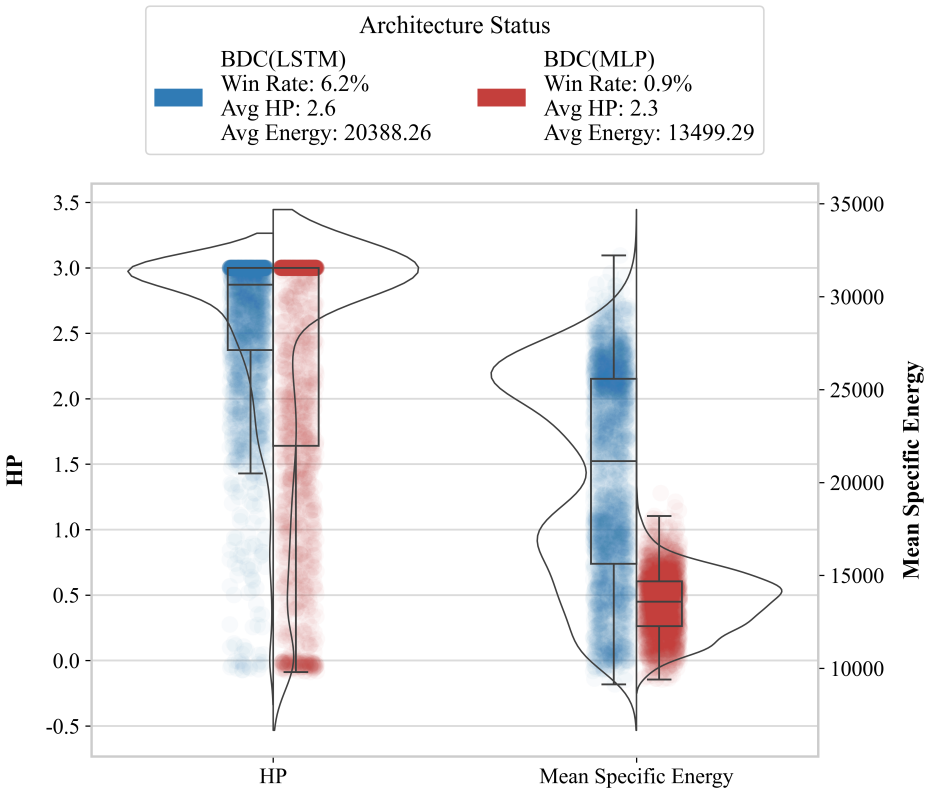


Fig. 11 Distributions of remaining health points (left) and specific energy (right) in direct cross-play between BDC-LSTM and BDC-MLP. Violin width indicates data density. The embedded boxplot shows the median and interquartile range (IQR), with whiskers extending to 1.5 × IQR. Overlaid scatter points represent individual samples. Together, these elements visualize the distribution shape, dispersion, and sample-level variability of both metrics.

We can also present the detailed engagement results between BDC-LSTM and BDC-MLP. Since both are trained with the proposed method and therefore have similar performance, the proportion of draws in their direct encounters is high. To further analyze their differences, Figure 11 presents the distributions of remaining HP and specific energy in cross-play.

As shown in Figure 11, BDC-LSTM outperforms BDC-MLP in direct competition. Their win rates are 6.2% and 0.9%, respectively, and BDC-LSTM also attains a higher average remaining HP (2.6 vs. 2.3). Meanwhile, the average specific energy

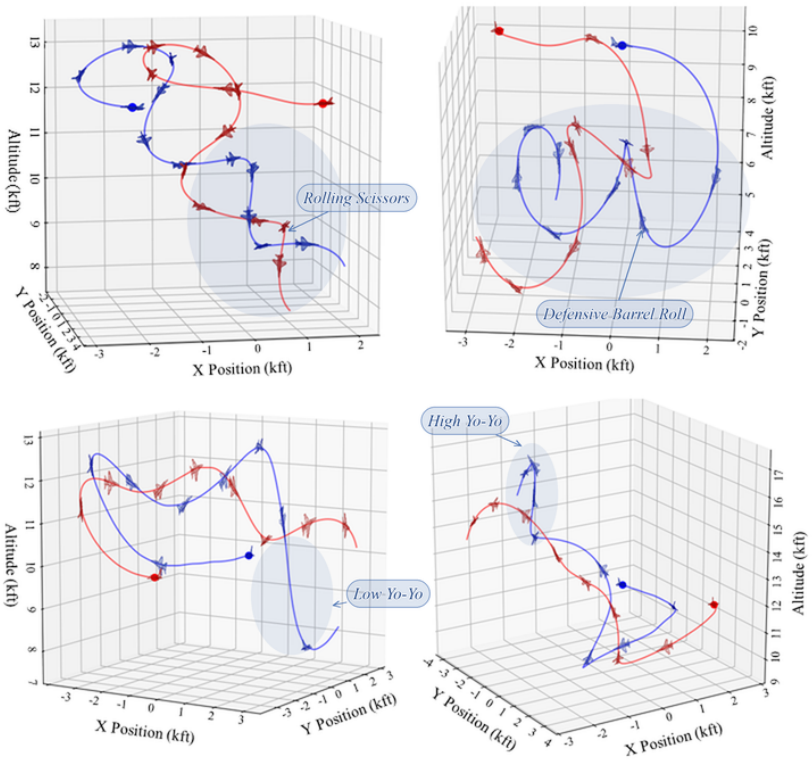


Fig. 12 Three-dimensional trajectory visualizations with 6-DOF aircraft attitudes during direct engagements between BDC-LSTM (blue) and BDC-MLP (red). Solid dots indicate the starting positions. The four panels show representative basic fighter maneuvers observed during the engagements: (a) rolling scissors, (b) defensive barrel roll, (c) low Yo-Yo, and (d) high Yo-Yo.

of BDC-LSTM is approximately 20388 ft, which is higher than the 13499 ft of BDC-MLP, indicating a larger energy margin over the opponent during the engagement, in agreement with the engagement-level energy margin of Table 7.

In addition, the two models exhibit different energy distribution patterns. BDC-MLP shows a concentrated unimodal distribution, whereas BDC-LSTM presents a bimodal characteristic. This suggests that the LSTM architecture may help the policy develop more differentiated energy-utilization modes under different engagement situations. These findings suggest that incorporating temporal memory may improve decision-making flexibility in adversarial combat.

4.4 Engagement Demonstration

To better analyze the agent's maneuvering decisions, we developed a three-dimensional trajectory visualization with six-degree-of-freedom state information (3D attitude plot). This section presents four representative engagement patterns observed in direct matches between BDC-LSTM (blue) and BDC-MLP (red).

Figure 12 illustrates four representative basic fighter maneuvers:

1. **Rolling Scissors**: As shown in Figure 12(a), after a close-range merge, both aircraft attempt to gain a favorable firing position behind the opponent, and the engagement develops into a typical rolling scissors maneuver.
2. **Defensive Barrel Roll**: As shown in Figure 12(b), the blue aircraft performs a defensive barrel roll while being pursued. By following a three-dimensional helical trajectory, it reduces the closure rate and changes the relative geometry, thereby inducing an overshoot by the red aircraft and improving its own tactical situation.
3. **Low Yo-Yo**: As shown in Figure 12(c), when the pursuit distance begins to increase, the blue aircraft executes a dive to gain speed, shortens its path on a lower turning plane, and compresses the relative distance before pulling up again to recover an attacking posture.
4. **High Yo-Yo**: As shown in Figure 12(d), when the blue aircraft enters the pursuit with a high closure rate, it does not simply follow the red aircraft in a planar turn. Instead, it deliberately climbs, converting part of its kinetic energy into potential energy, reducing the risk of overshoot and improving the turning geometry before re-entering a favorable firing position.

These representative cases suggest that the BDC method is able to spontaneously develop diverse maneuvering strategies according to relative position, speed, and energy state.

4.5 Opponent Pool Initialization Ablation

The dynamic opponent pool of Section 3.3 combines a protected initialization with a set of dynamic slots that are periodically refreshed. To isolate the contribution of the initialization, we compare two arms that differ only in whether the pool is seeded with externally pretrained opponents. In the with-initialization arm the pool is seeded with a goal-point navigation policy together with two combat-capable policies derived from it, all three protected, which leaves one refreshable slot and reproduces the configuration used by the proposed method. In the without-initialization arm the pool initially contains a single model, namely the arm's own stage-1 checkpoint, and no protected opponent, so that all four slots are refreshable from the first update onwards. Everything else is held fixed. Each arm is trained for 3.0×10^8 steps, and is then evaluated on the benchmark pool of Section 4.3.1 with 1000 engagements per opponent and randomized initial states.

Figure 13 shows the evolution of the evaluation return during the battle stage. The two arms differ both in how long the return keeps improving and in the level at which it settles. With initialization the return rises steadily through the first part of training, from 195 in the 10^7-step window ending at 1.5×10^7 steps to about 1250 at 6.5×10^7 steps, that is by a factor of more than six over the first 8×10^7 steps, and it stays at a comparable or higher level afterwards, oscillating between roughly 400 and 1300 without a further systematic trend. Without initialization the return reaches 539 in the window ending at 2×10^7 steps, which remains its highest value of the whole run, and it shows no further improvement after 5×10^7 steps: the average over the windows covering 5×10^7 to 1.0×10^8 steps is 350, below the average of 390 over the

first five windows, and the return declines further towards the end of training. Once training has settled, the two arms therefore differ in level as well as in the duration of the improvement: over the last 10^8 steps, from 2.1×10^8 to 3.0×10^8 steps, the mean evaluation return is 744 with initialization against 275 without, a factor of about 2.7.

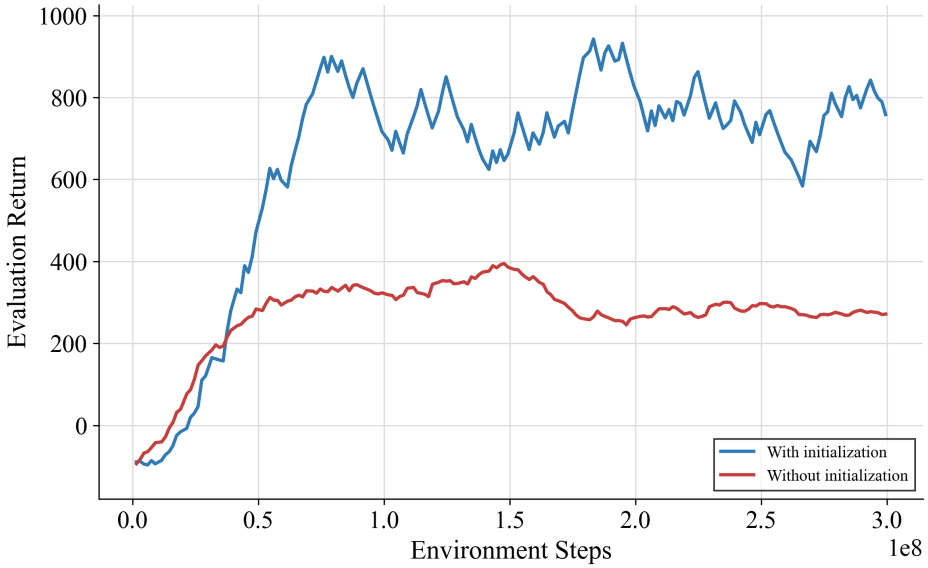


Fig. 13 Training stability of the two arms over the 3.0×10^8 steps of the battle stage, showing the exponentially smoothed evaluation return. The curves start after the stage-1 pretraining, which is identical in the two arms. Without external initialization the return stops improving after the first few cycles and settles at a markedly lower level than with initialization.

Table 8 reports the outcome distribution and the air-combat metrics on the benchmark.

Table 8 Effect of opponent pool initialization on the benchmark of Section 4.3.1 (8×1000 engagements per arm). ΔHP is the mean difference in remaining health points between the ego and the opponent at the end of an engagement.

Arm	Win rate	Draw	Loss	Gun opportunity	ΔHP
With initialization	42.49%	57.2%	0.26%	0.156	+1.80
Without initialization	14.44%	84.1%	1.48%	0.078	+0.99

The two arms differ by 28.1 percentage points in win rate, and the same separation appears in every secondary metric: the gun-opportunity ratio, that is the fraction of the engagement time during which the opponent remains inside the weapon engagement zone, falls from 0.156 to 0.078, the mean health margin at the end of an

engagement falls from +1.80 to +0.99, and the mean total damage inflicted on the opponent falls from 2.02 to 1.32.

Table 9 Win rate of each arm against the four opponent types of the benchmark pool, which contains four level-flight, two early-combat, one goal-point and one weak-reference opponent.

Arm	Level flight	Early combat	Goal point	Weak reference
With initialization	66.6%	30.6%	5.0%	7.3%
Without initialization	25.1%	6.7%	1.6%	0.1%

The breakdown by opponent type (Table 9) localizes the failure. Against the four non-manoeuvring level-flight opponents the without-initialization arm still wins 25.1% of the engagements, but its win rate falls to 6.7% against the two early combat opponents and to 1.6% against the goal-point policy, that is, precisely against the opponents that manoeuvre in order to deny a firing solution; with initialization the same sequence only degrades from 66.6% to 30.6% and 5.0%. The degradation is thus concentrated on the opponents that manoeuvre rather than being uniform across the pool, and the per-opponent win rates of Figure 14 show the same pattern on every individual opponent. A comparison between the two arms alone points in the same direction without involving the benchmark: over 300 engagements in each direction, the with-initialization arm wins 4.0% of the engagements and loses none, with a mean health margin of +0.81, whereas the without-initialization arm wins none of the 300 engagements in the opposite direction (0.0%), finishes at −0.83 health points and obtains only about one seventh of the gun-opportunity time of its opponent.

The refresh statistics of the pool provide a direct view of the adversarial signal. Every refresh of the pool is decided by an evaluation in which the current snapshot is matched against the pool members. In the without-initialization arm, 108 such comparisons were performed during training; the challenger achieved a mean win rate of 0.26%, 99.7% of the engagements ended in a draw, and only four comparisons contained any engagement that the challenger won. The pool was nevertheless refreshed 27 times, because the update rule also rewards small differences in remaining health, whose mean value was +0.03 health points. These numbers are consistent with a pool that progressively loses its ability to discriminate between policies, and with a policy that settles into an equilibrium in which engagements rarely produce a decision: the final pool of that arm plays 100% draws against itself, with almost identical remaining health on both sides (2.74 against 2.77) and a gun-opportunity ratio of 1.9%. In the with-initialization arm, by contrast, the three protected opponents remain external throughout training, and its single refreshable slot stopped being replaced only in the last four cycles.

Overall, the results of this ablation indicate that the initialization of the opponent pool has a substantial influence on the outcome of training in the setting studied here. When the pool contains only the agent's own early checkpoint, its refresh statistics suggest that the pool rarely produces a decisive comparison between policies, the

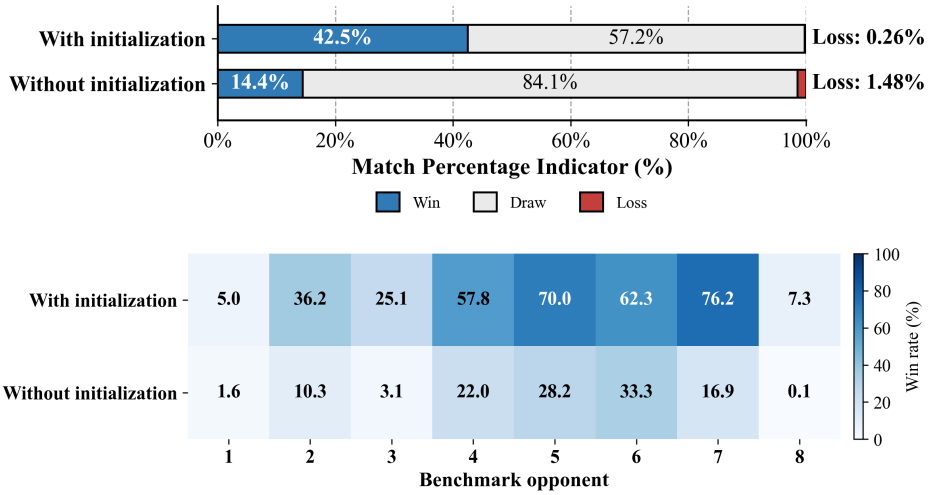


Fig. 14 Benchmark outcomes without and with opponent pool initialization. Top: win, draw and loss distribution over the eight benchmark opponents. Bottom: per-opponent win rate, with the opponents numbered 1–8 in the order of the benchmark pool: one goal-point policy, two early-combat policies, four level-flight policies and one weak reference policy.

evaluation return ceases to improve early and settles at a markedly lower level, and the policy obtained at the end of training performs considerably worse on the benchmark, in particular against the opponents that manoeuvre. Curriculum learning and the periodic refresh both operate on top of the pool and, in this experiment, they did not compensate for the absence of an external anchor.

5 Conclusion

This paper proposes a reinforcement learning framework for within-visual-range autonomous air combat, providing a solution with generalization and robustness. Through the integration of curriculum learning and dynamic opponent pool, the proposed framework stabilizes training and mitigates cyclic dominance. Experimental evaluations indicate that the proposed method yields competitive performance against baseline methods in terms of win rate and of the energy margin that it holds over the opponent. Ultimately, this work offers a viable baseline for addressing flight control and tactical decision-making in autonomous dogfighting.

Appendix A

The appendix provides supplementary implementation details for the proposed training framework. Table A1 summarizes the complete state observation space together with their corresponding physical lower and upper bounds. Table A2 lists the basic reward terms used for training the level-flight opponents, where $\sigma(\cdot)$ denotes the logistic function; u^*_{ail}, u^*_{thr}, and ϕ^* denote the target values of aileron input, throttle input, and roll angle, respectively; h_{low} and h_{high} denote the lower and upper boundaries of

the allowable flight envelope; $k_a, k_t, k_r, k_l, k_h, k_s, k_{\Delta h}$ are the weighting coefficients of the corresponding reward terms; and s_l, s_h control the slope of the boundary penalty transition. Table A3 reports the main algorithm and network hyperparameters used for both the LSTM-based and MLP-based policies. Table A4 presents the environment and stage configuration.

Table A1 Complete state observation space variables and extremum boundaries.

State category	Variable name	Lower bound	Upper bound
Aircraft base states	h	-1400 ft	85000 ft
	θ	$-\pi/2$ rad	$\pi/2$ rad
	ϕ	$-\pi$ rad	π rad
	u	-2200 ft/s	2200 ft/s
	v	-2200 ft/s	2200 ft/s
	w	-2200 ft/s	2200 ft/s
	p	-2π rad/s	2π rad/s
	q	-2π rad/s	2π rad/s
	r	-2π rad/s	2π rad/s
Aircraft tracking states	heading	0 deg	360 deg
	a_x, a_y, a_z	–	–
	$a_{roll}, a_{pitch}, a_{yaw}$	–	–
	α	-180 deg	180 deg
	β	-180 deg	180 deg
	aileron, elevator, rudder	-1	1
	throttle Aug, engine thrust	–	–
	vtrue	0 ft/s	2200 ft/s
Relative geometry	D	0 ft	500000 ft
	θ_{tra}	$-\pi$ rad	π rad
	θ_{bea}, accounting roll/pitch	$-\pi$ rad	π rad
	θ_{ele}, accounting roll/pitch	$-\pi$ rad	π rad
	θ_{bea}, point mass	$-\pi$ rad	π rad
	θ_{ele}, point mass	$-\pi$ rad	π rad
	θ_{adv}	$-\pi$ rad	π rad
	V_c	-100000 ft/s	100000 ft/s
Opponent states	x_{oppo}, y_{oppo}	-100000 ft	100000 ft
	h_{oppo}	-1400 ft	85000 ft
	$\phi_{oppo}, \theta_{oppo}$	$-\pi, -\pi/2$ rad	$\pi, \pi/2$ rad
	heading$_{oppo}$	0 deg	360 deg
	$u_{oppo}, v_{oppo}, w_{oppo}$	-2200 ft/s	2200 ft/s
	$p_{oppo}, q_{oppo}, r_{oppo}$	-2π rad/s	2π rad/s
	$\alpha_{oppo}, \beta_{oppo}$	-180 deg	180 deg
	$\theta_{tra,oppo}$	$-\pi$ rad	π rad
	vtrue$_{oppo}$	0 ft/s	2200 ft/s
	$\theta_{bea,oppo}, \theta_{ele,oppo}$	$-\pi$ rad	π rad
Historical actions	$\delta_a, \delta_e, \delta_r, \delta_T$	-1	1

Table A2 Basic reward terms used for training the level-flight opponents.

Reward term	Mathematical form
Aileron magnitude penalty	$r_{\text{ail}} = -k_a \lvert u_{\text{ail}} - u^*_{\text{ail}} \rvert$
Throttle penalty	$r_{\text{thr}} = -k_t \lvert u_{\text{thr}} - u^*_{\text{thr}} \rvert$
Roll attitude penalty	$r_{\text{roll}} = -k_r \lvert \phi - \phi^* \rvert$
Lower altitude penalty	$r_{\text{low}} = -k_l \left(1 - \sigma\left(\frac{h - h_{\text{low}}}{s_l}\right)\right)$
Upper altitude penalty	$r_{\text{high}} = -k_h \left(1 - \sigma\left(\frac{h_{\text{high}} - h}{s_h}\right)\right)$
Action smoothness penalty	$r_{\text{smooth}} = -k_s \sum_{i \in \{\text{ail,ele,rud}\}} \lvert u_i^{(t)} - u_i^{(t-1)} \rvert$
Altitude smoothness penalty	$r_{\Delta h} = -k_{\Delta h} \lvert h_t - h_{t-1} \rvert$

Table A3 Algorithm and network hyperparameters.

Hyperparameter	LSTM	MLP
Learning rate	5.0×10^{-5}	1.0×10^{-4}
Rollout steps	2048	2048
Batch size	128	128
Epochs	10	10
Gamma (γ)	0.99	0.99
GAE lambda (λ)	0.95	0.95
Clip range	0.2	0.2
Value function coefficient	0.5	0.5
Max grad norm	0.5	0.5
Share features extractor	False	True
LSTM hidden size	256	–
Network arch (policy)	[512, 512]	[12288]
Network arch (value)	[512, 512]	Shared

References

[1] Strand, A., Gorton, P.R., Brathen, K.: Modeling air combat behavior for simulation-based pilot training: a survey of machine learning approaches. IEEE Access **13**, 113666–113685 (2025) https://doi.org/10.1109/ACCESS.2025.3580279

[2] Pope, A.P., Ide, J.S., Mićović, D., Diaz, H., Twedt, J.C., Alcedo, K., Walker, T.T., Rosenbluth, D., Ritholtz, L., Javorsek, D.: Hierarchical reinforcement learning for air combat at DARPA's AlphaDogfight trials. IEEE Transactions on Artificial Intelligence **4**(6), 1371–1385 (2023) https://doi.org/10.1109/TAI.2022.3222143

[3] Koch, W., Mancuso, R., West, R., Bestavros, A.: Reinforcement learning for UAV attitude control. ACM Trans. Cyber-Phys. Syst. **3**(2) (2019) https://doi.org/10.1145/3301273

Table A4 Environment and stage configuration.

Parameter	Value
Aircraft model	F-16
Observation clipping threshold	10.0
Random seed	42
Number of CPUs	30
Evaluation frequency	50,000 episodes
Battle step	10,000,000
Battle cycles	30
Number of evaluation episodes	50
Threshold timesteps	20,000,000
Update frequency	1,000 episodes

[4] Bøhn, E., Coates, E.M., Reinhardt, D., Johansen, T.A.: Data-efficient deep reinforcement learning for attitude control of fixed-wing UAVs: field experiments. IEEE Transactions on Neural Networks and Learning Systems **35**(3), 3168–3180 (2024) https://doi.org/10.1109/TNNLS.2023.3263430

[5] O'Connell, M., Shi, G., Shi, X., Azizzadenesheli, K., Anandkumar, A., Yue, Y., Chung, S.-J.: Neural-Fly enables rapid learning for agile flight in strong winds. Science Robotics **7**(66), 6597 (2022) https://doi.org/10.1126/scirobotics.abm6597

[6] Clarke, S.G., Hwang, I.: Deep reinforcement learning control for aerobatic maneuvering of agile fixed-wing aircraft. In: AIAA Scitech 2020 Forum. AIAA, Orlando, FL, USA (2020). https://doi.org/10.2514/6.2020-0136

[7] Clarke, R., Fletcher, L., East, S., Richardson, T.: Reinforcement learning derived high-alpha aerobatic manoeuvres for fixed wing operation in confined spaces. Algorithms **16**(8), 384 (2023) https://doi.org/10.3390/a16080384

[8] Chai, J., Chen, W., Zhu, Y., Yao, Z.-X., Zhao, D.: A hierarchical deep reinforcement learning framework for 6-DOF UCAV air-to-air combat. IEEE Transactions on Systems, Man, and Cybernetics: Systems **53**(9), 5417–5429 (2023) https://doi.org/10.1109/TSMC.2023.3270444

[9] Wang, Y., Li, H., Shen, Q.: A hierarchical multi-task and multi-agent assignment approach: learning DQN strategy from execution. IEEE Transactions on Automation Science and Engineering **22**, 14712–14722 (2025) https://doi.org/10.1109/TASE.2025.3560121

[10] Feng, X., Ma, Y., Zhao, L., Yang, H.: Autonomous decision making with reinforcement learning in multi-UAV air combat. In: 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC), pp. 2874–2879. IEEE, Honolulu, HI, USA (2023). https://doi.org/10.1109/SMC53992.2023.10394123

[11] Selmonaj, A., Szehr, O., Del Rio, G., Antonucci, A., Schneider, A., Rüegsegger, M.: Hierarchical multi-agent reinforcement learning for air combat maneuvering. In: 2023 International Conference on Machine Learning and Applications (ICMLA), pp. 1031–1038 (2023). https://doi.org/10.1109/ICMLA58977.2023.00153

[12] Chen, Y.-H., Chiang, M.-H., Peng, C.-C.: Physics-informed reinforcement learning for air-to-air F16 target aiming. Advanced Engineering Informatics **69**, 103988 (2026) https://doi.org/10.1016/j.aei.2025.103988

[13] Berndt, J.S., Coconnier, B., De Marco, A., McLeod, S.: JSBSim - an Open Source Flight Dynamics Software Library. https://doi.org/10.5281/zenodo.20258621 . https://github.com/JSBSim-Team/jsbsim

[14] Li, B.-W., Tan, T., Li, J., Zhang, J.-W., Zhang, X.-R.: A progressive deep reinforcement learning approach for 6-DOF air combat decision-making. Journal of Computer Applications (2025) https://doi.org/10.19678/j.issn.1000-3428.0252480 . Online published

[15] Giral, F., Gomez, I., Le Clainche, S.: Intercepting unauthorized aerial robots in controlled airspace using reinforcement learning. Aerospace Science and Technology **168**, 110985 (2026) https://doi.org/10.1016/j.ast.2025.110985

[16] Yu, L., Li, Z., Ansari, N., Sun, X.: Hybrid transformer based multi-agent reinforcement learning for multiple unpiloted aerial vehicle coordination in air corridors. IEEE Transactions on Mobile Computing **24**(6), 5482–5495 (2025) https://doi.org/10.1109/TMC.2025.3532204

[17] Yang, Q., Li, D., Huang, J., Zhou, P., Zhang, S.: Close-range air combat autonomous maneuver decision-making of UAV based on TD3-Transformer. International Journal of Aeronautical and Space Sciences (2025) https://doi.org/10.1007/s42405-025-01079-2

[18] Hsu, C.D., Jeong, H., Pappas, G.J., Chaudhari, P.: Scalable reinforcement learning policies for multi-agent control. In: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 4785–4791. IEEE, Prague, Czech Republic (2021). https://doi.org/10.1109/IROS51168.2021.9636344

[19] Bae, J.H., Jung, H., Kim, S., Kim, S., Kim, Y.-D.: Deep reinforcement learning-based air-to-air combat maneuver generation in a realistic environment. IEEE Access **11**, 26427–26440 (2023) https://doi.org/10.1109/ACCESS.2023.3257849

[20] Wang, L., Zheng, S., Tai, S., Liu, H., Yue, T.: UAV air combat autonomous trajectory planning method based on robust adversarial reinforcement learning. Aerospace Science and Technology **153**, 109402 (2024) https://doi.org/10.1016/j.ast.2024.109402

[21] Li, S., Zuo, R., Liu, B., Liu, P., Zhao, Y.: An imitative reinforcement learning framework for autonomous dogfight. Preprint at https://arxiv.org/abs/2406.11562 (2025)

[22] Kong, W.-r., Zhou, D.-y., Du, Y.-j., Zhou, Y., Zhao, Y.-y.: Hierarchical multi-agent reinforcement learning for multi-aircraft close-range air combat. IET Control Theory & Applications **17**(13), 1840–1862 (2023) https://doi.org/10.1049/cth2.12413

[23] Han, Y., Piao, H., Hou, Y., Sun, Y., Sun, Z., Zhou, D., Yang, S., Peng, X., Fan, S.: Deep relationship graph reinforcement learning for multi-aircraft air combat. In: 2022 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. IEEE, Padua, Italy (2022). https://doi.org/10.1109/IJCNN55064.2022.9892208

[24] Selmonaj, A., Szehr, O., Del Rio, G., Antonucci, A., Schneider, A., Rüegsegger, M.: Hierarchical multi-agent reinforcement learning for air combat maneuvering. In: 2023 International Conference on Machine Learning and Applications (ICMLA), pp. 1031–1038. IEEE, Jacksonville, FL, USA (2023). https://doi.org/10.1109/ICMLA58977.2023.00153

[25] Han, H., Cheng, J., Lv, M., Duan, H.: Augmenting the robustness of tactical maneuver decision-making in unmanned aerial combat vehicles during dogfights via prioritized population play with diversified partners. IEEE Transactions on Aerospace and Electronic Systems **61**(5), 12892–12907 (2025) https://doi.org/10.1109/TAES.2025.3578294

[26] Chen, C., Song, T., Mo, L., Lv, M., Lin, D.: Autonomous dogfight decision-making for air combat based on reinforcement learning with automatic opponent sampling. Aerospace **12**(3), 265 (2025) https://doi.org/10.3390/aerospace12030265

[27] Gorren: Gym-jsbsim, v1.0.0, GitHub. https://github.com/Gor-Ren/gym-jsbsim

[28] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. Preprint at https://arxiv.org/abs/1707.06347 (2017)

[29] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N.: Stable-Baselines3: reliable reinforcement learning implementations. Journal of Machine Learning Research **22**(268), 1–8 (2021)

[30] Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997) https://doi.org/10.1162/neco.1997.9.8.1735

[31] Neufeld, J., Watson, George M. Jr, Chenoweth, D. (eds.): Technology and the Air Force: a Retrospective Assessment. Air Force History and Museums Program, United States Air Force, Washington, DC (1997)

[32] Boyd, J.R., Christie, T.P., Gibson, J.E.: Energy-maneuverability. Air Proving Ground Center Report APGC-TR-66-4 (1966)